

06026240 INTELLIGENT SYSTEM DEVELOPMENT

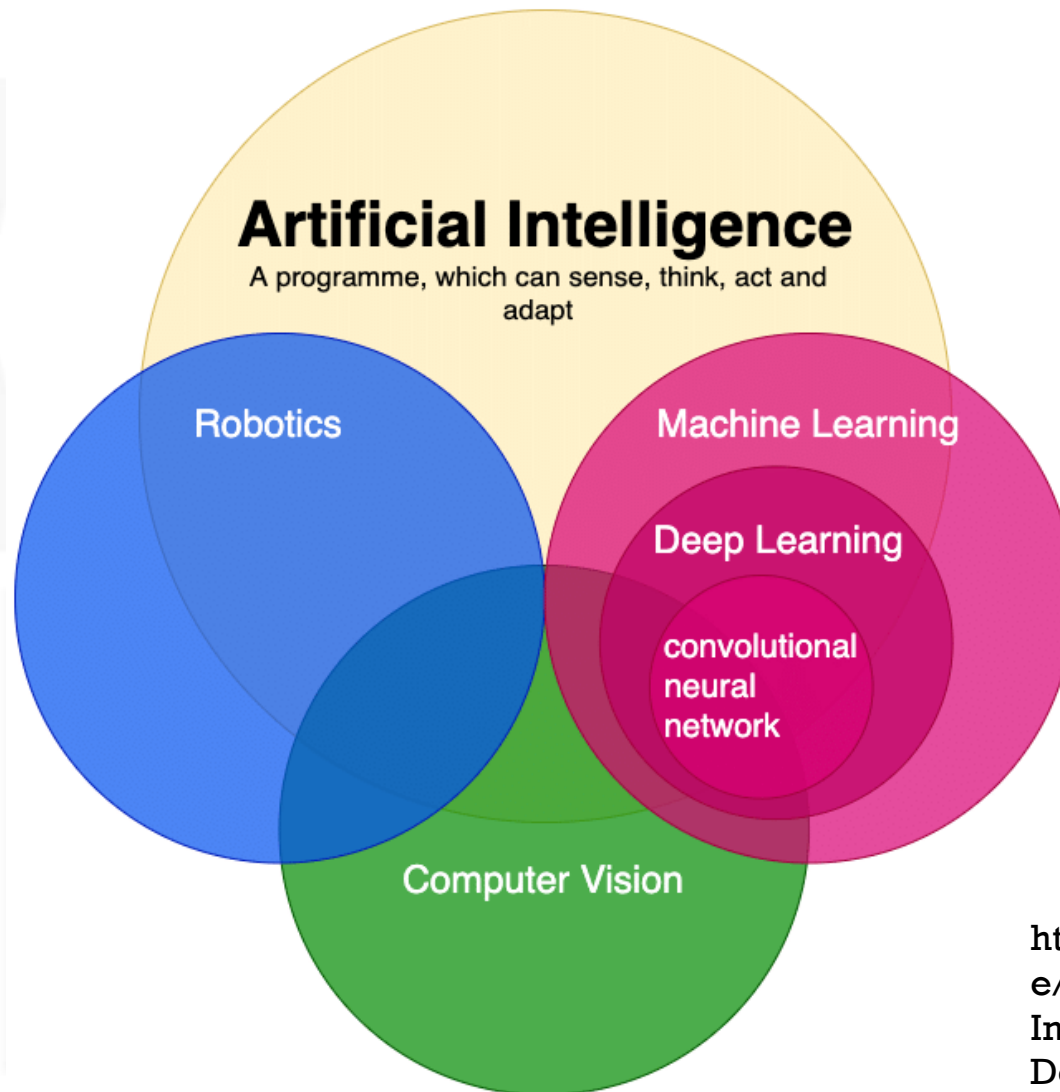
Chapter III

Introduction to ML and Deep learning

with Dr. Bhattarabhorn Wattanacheep

Is Computer Vision Artificial Intelligence?

- Relation between Artificial Intelligence, Machine Learning and Deep Learning, Computer Vision.



https://www.researchgate.net/figure/Relationbetween-Artificial-Intelligence-Machine-Learning-and-Deep-Learning-Computer_fig1_342978934

What is Deep Learning?

ARTIFICIAL INTELLIGENCE

Any technique that enables computers to mimic human behavior



MACHINE LEARNING

Ability to learn without explicitly being programmed



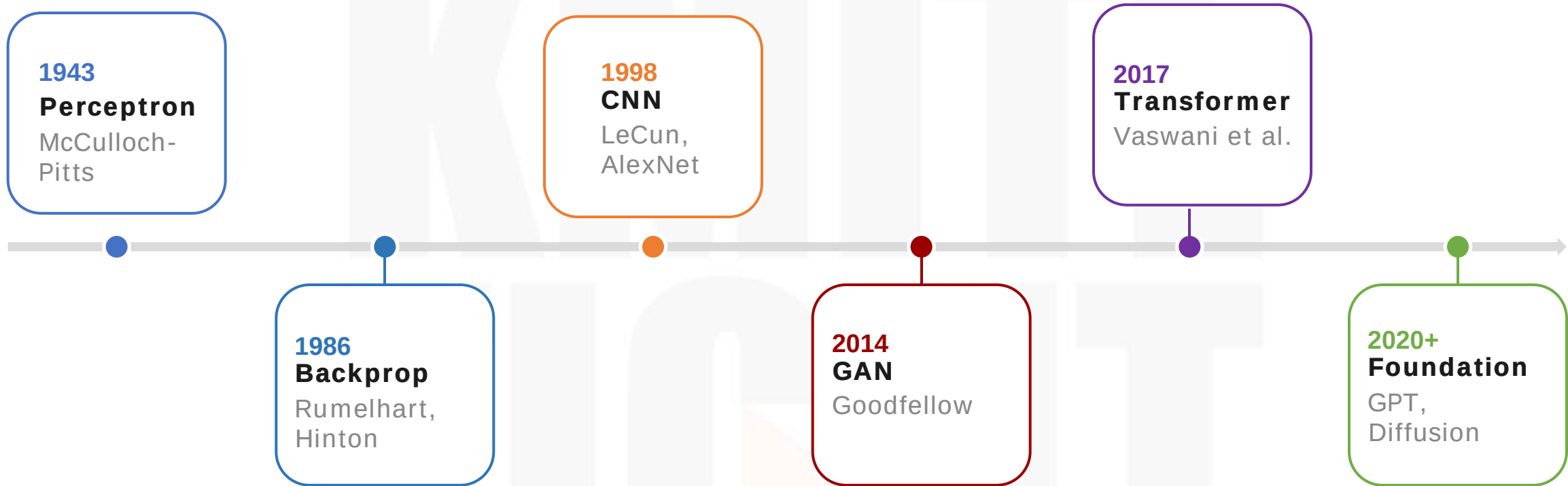
DEEP LEARNING

Extract patterns from data using neural networks

3 1 3 4 7 2
1 7 4 2 3 5

Teaching computers how to **learn a task** directly from **raw data**

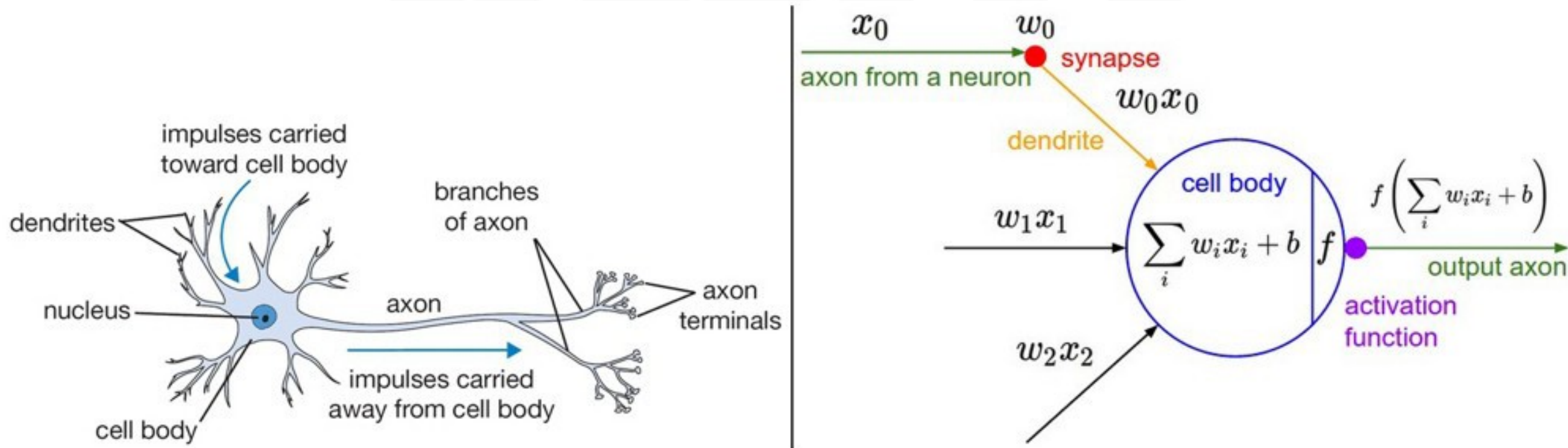
Evolution of Neural Networks



Key Insight: Transformer (2017) เปลี่ยนทุกอย่าง — เป็นพื้นฐานของ GPT, BERT, Stable Diffusion และ AI สมัยใหม่ทั้งหมด

Neural Networks Basics

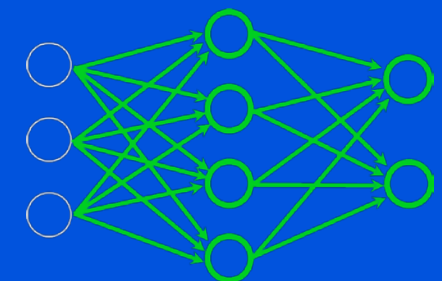
Inspiration from Biology



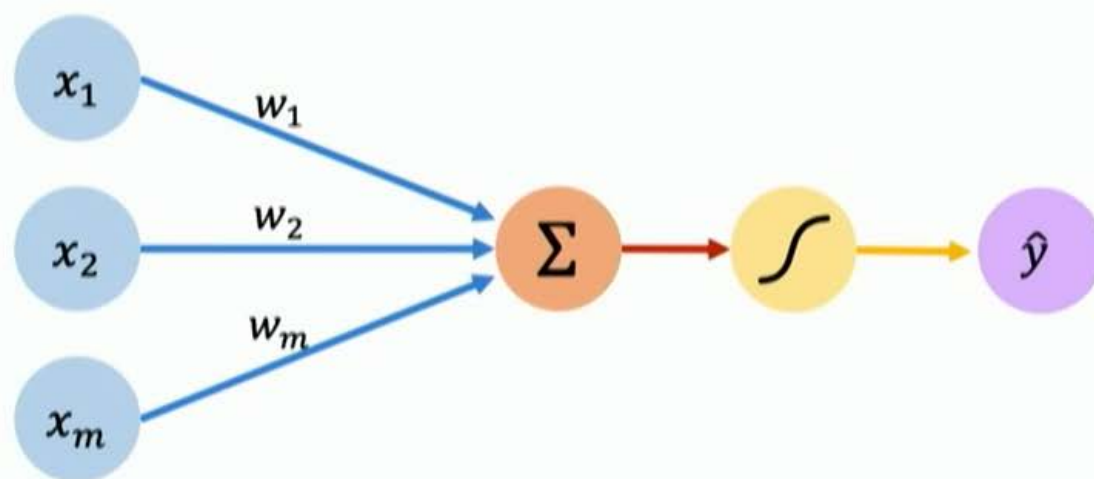
A cartoon drawing of a biological neuron (left) and its mathematical model (right).

The Perceptron

The structural building block of deep learning



The Perceptron: Forward Propagation



Inputs Weights Sum Non-Linearity Output

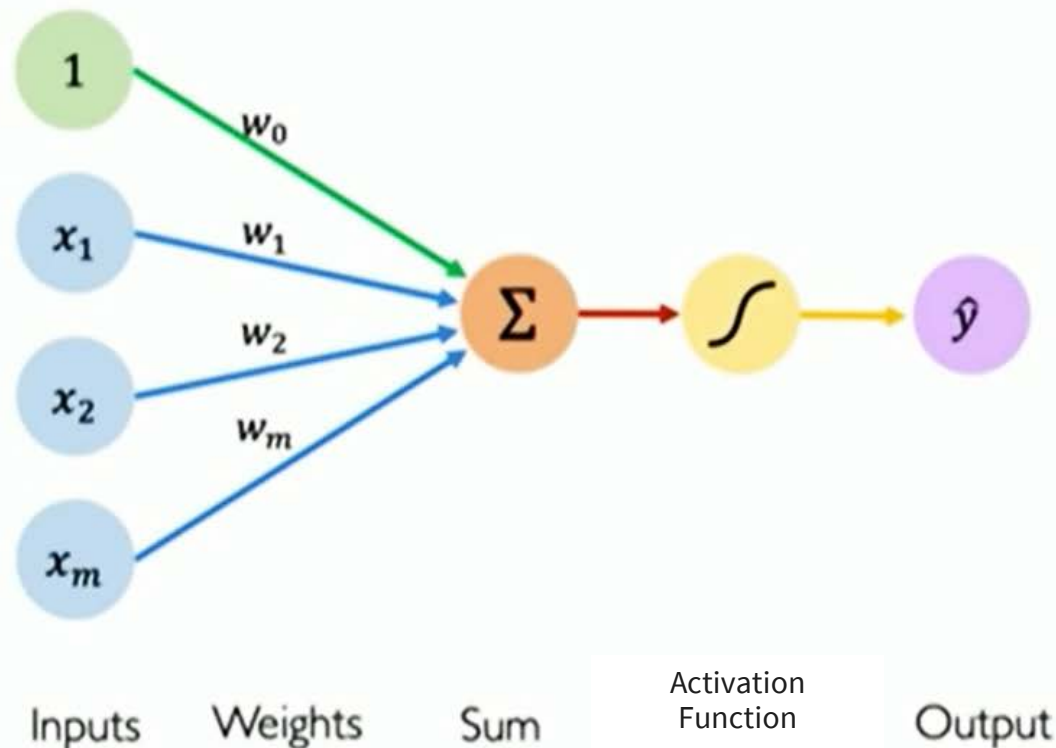
Output

Linear combination of inputs

$$\hat{y} = g \left(\sum_{i=1}^m x_i w_i \right)$$

Non-linear activation function

The Perceptron: Forward Propagation



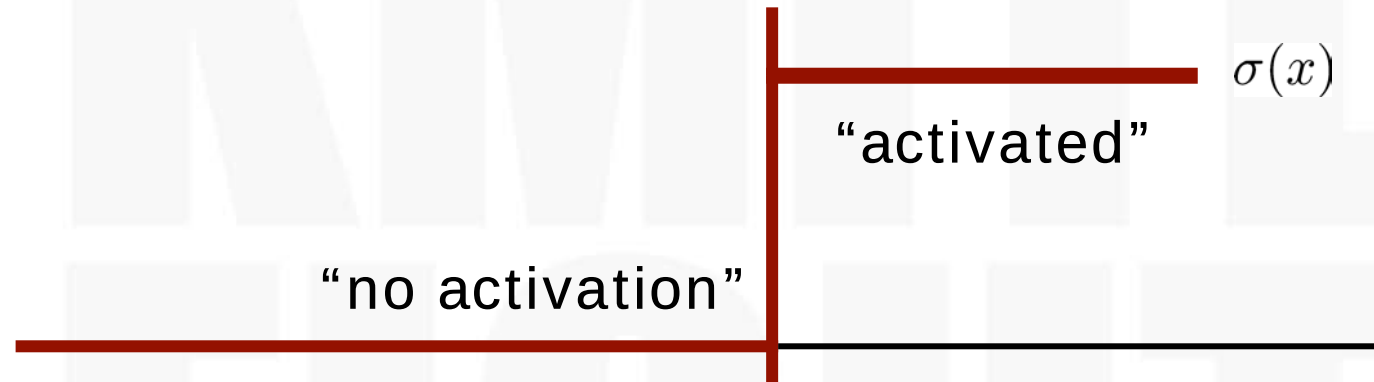
$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

$$\hat{y} = g (w_0 + \mathbf{X}^T \mathbf{W})$$

$$\text{where: } \mathbf{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \text{ and } \mathbf{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

Activation Function

- Biological neurons are either active or not active
- We can see this as a step function:

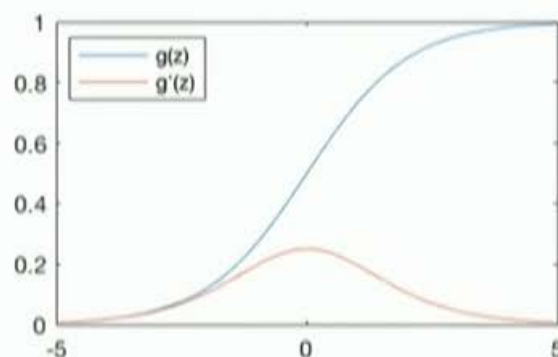


- Bias tells us where the activation happens
- We can model this behavior through

$$a = \begin{cases} 0 & \sum_i w_i a_i \leq -b \\ 1 & \text{otherwise} \end{cases}$$

Common Activation Functions

Sigmoid Function

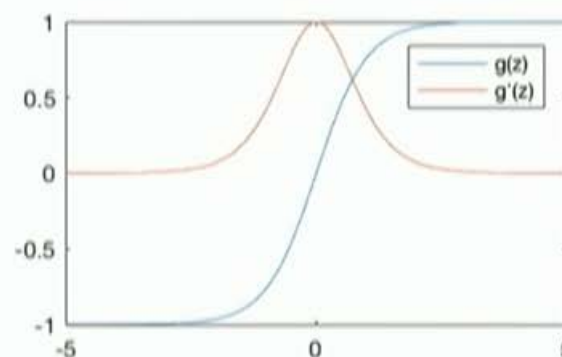


$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

`tf.math.sigmoid(z)`

Hyperbolic Tangent

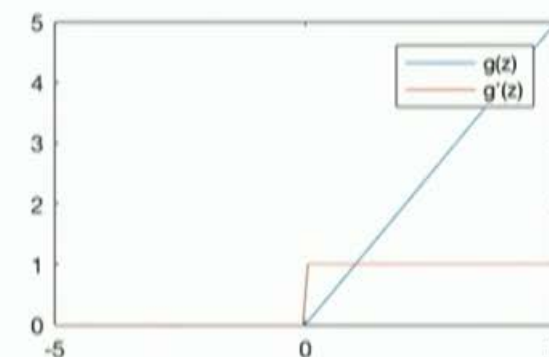


$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

`tf.math.tanh(z)`

Rectified Linear Unit (ReLU)



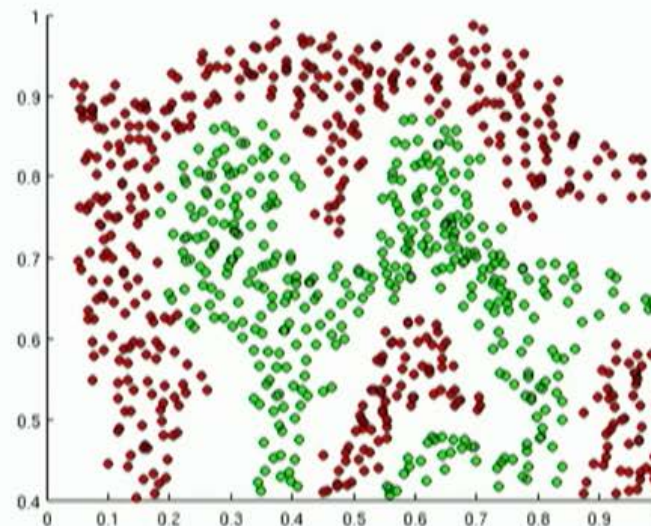
$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

`tf.nn.relu(z)`

Importance of Activation Functions

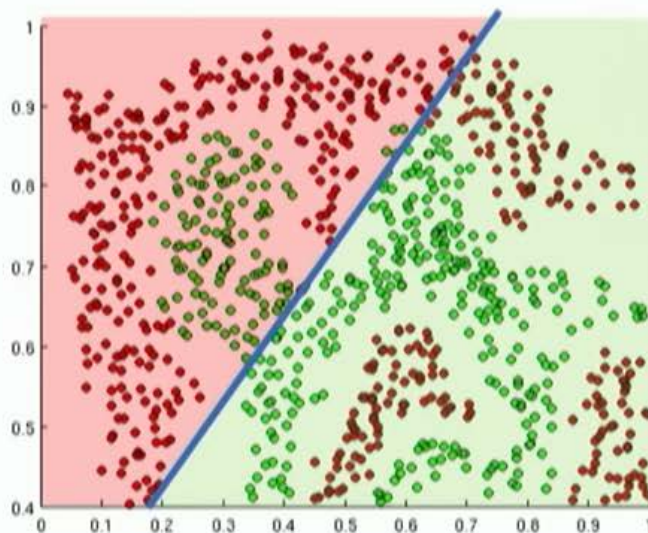
*The purpose of activation functions is to **introduce non-linearities** into the network*



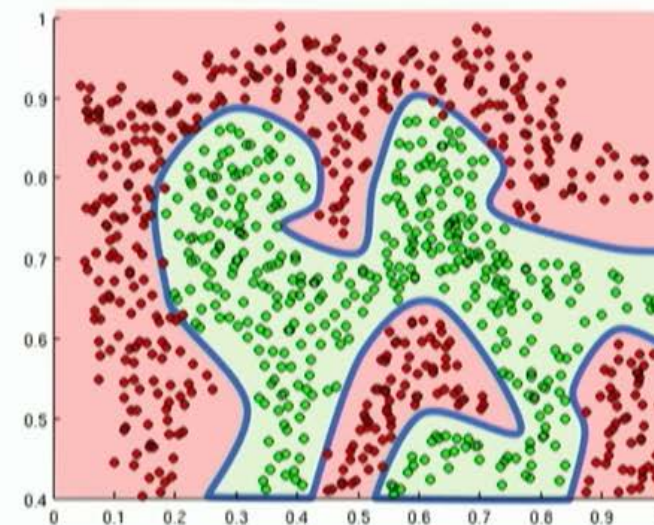
What if we wanted to build a neural network to distinguish green vs red points?

Importance of Activation Functions

The purpose of activation functions is to **introduce non-linearities** into the network

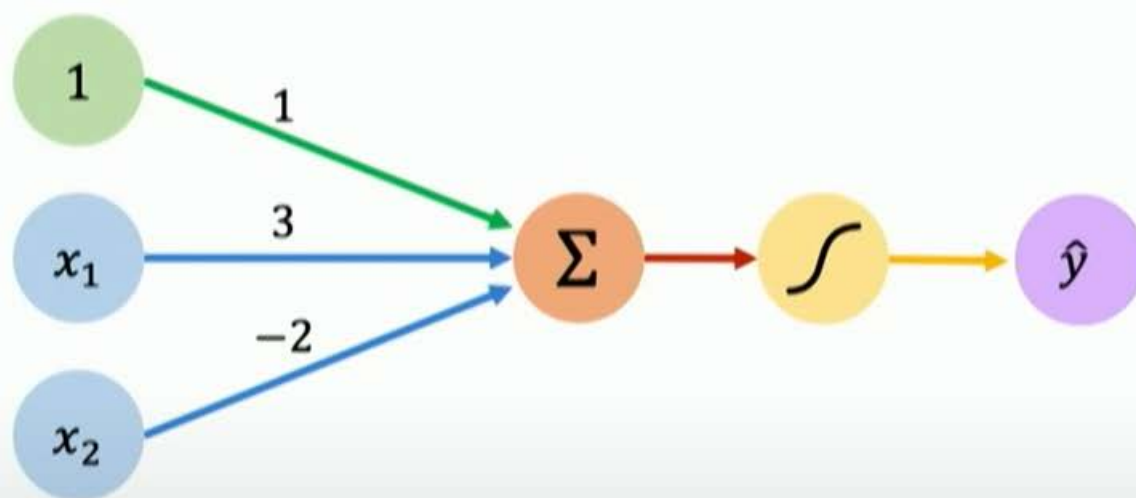


Linear activation functions produce linear decisions no matter the network size



Non-linearities allow us to approximate arbitrarily complex functions

The Perceptron: Example

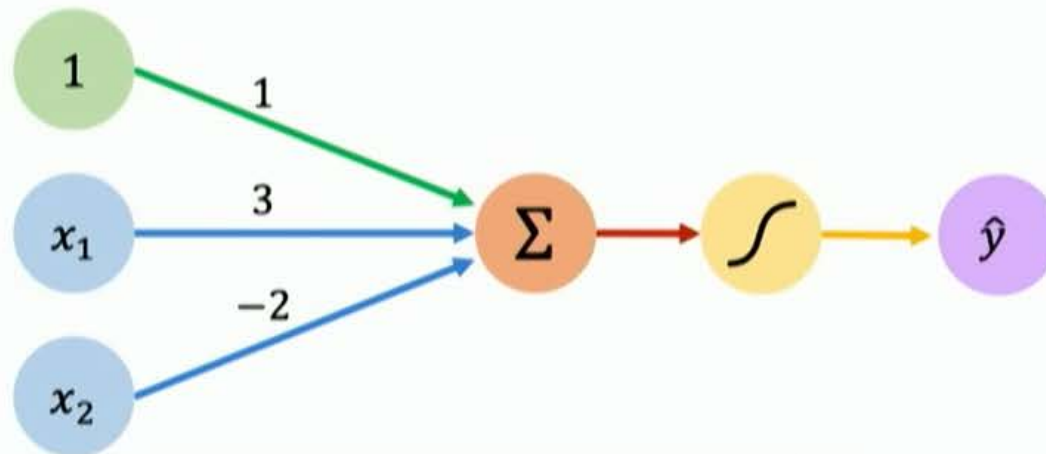


We have: $w_0 = 1$ and $\mathbf{W} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$

$$\begin{aligned}\hat{y} &= g(w_0 + \mathbf{X}^T \mathbf{W}) \\ &= g\left(1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix}\right) \\ \hat{y} &= g(1 + 3x_1 - 2x_2)\end{aligned}$$

This is just a line in 2D!

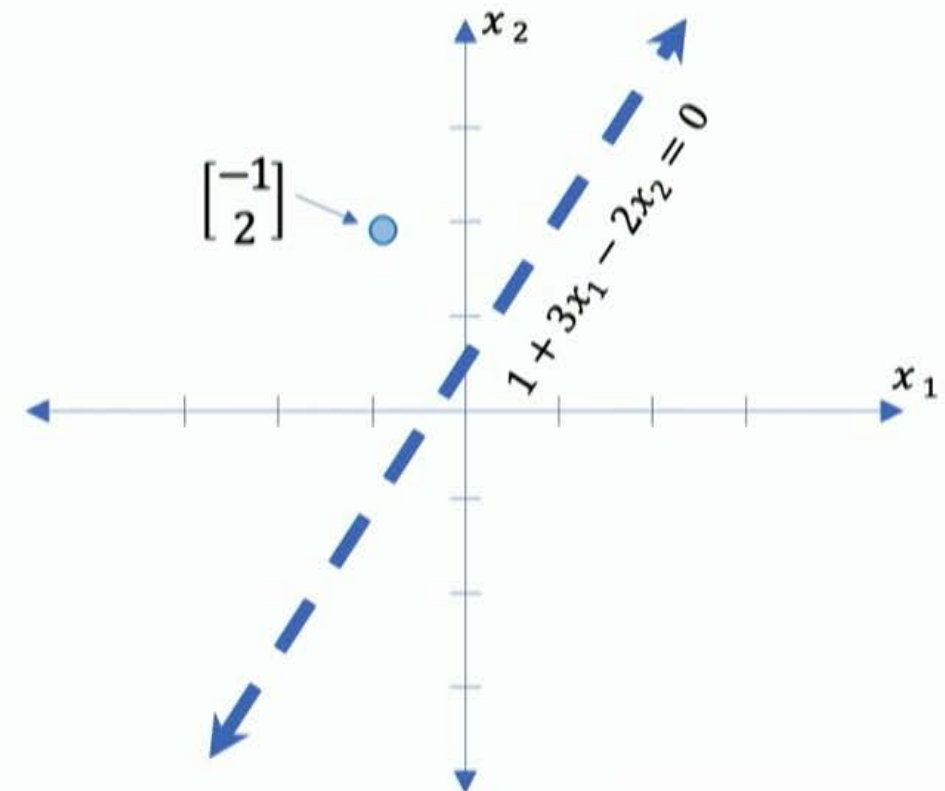
The Perceptron: Example



Assume we have input: $\mathbf{X} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$

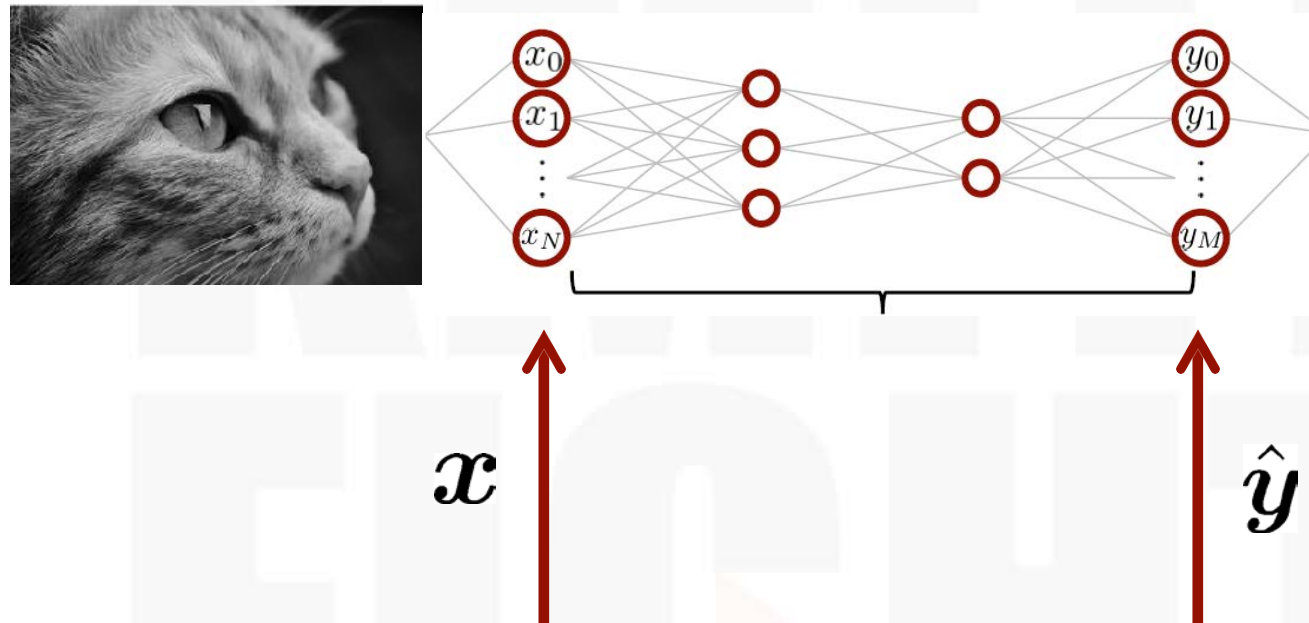
$$\begin{aligned}\hat{y} &= g(1 + (3 * -1) - (2 * 2)) \\ &= g(-6) \approx 0.002\end{aligned}$$

$$\hat{y} = g(1 + 3x_1 - 2x_2)$$



Building Neural Networks with Perceptrons

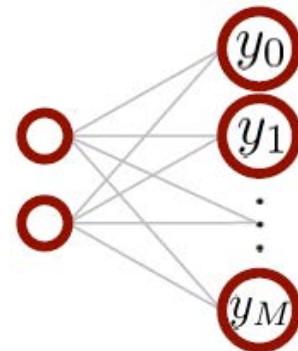
Image Classification



pixels intensities
are the values of
the input layer

output layer is a
vector of likelihoods
for the possible labels

What is the Network's Output?



Is it a...
cat or a
dog or a
human or a
...?

$$\begin{bmatrix} 98\% \\ 1\% \\ 0.1\% \\ \vdots \end{bmatrix}$$

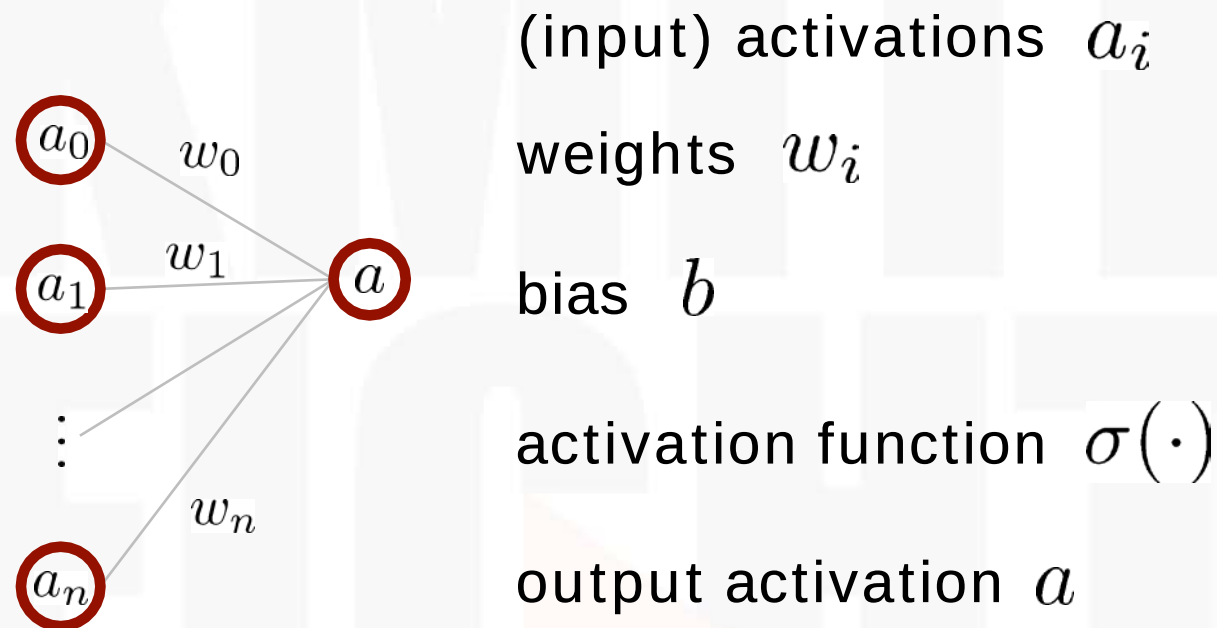
“cat”



largest
value

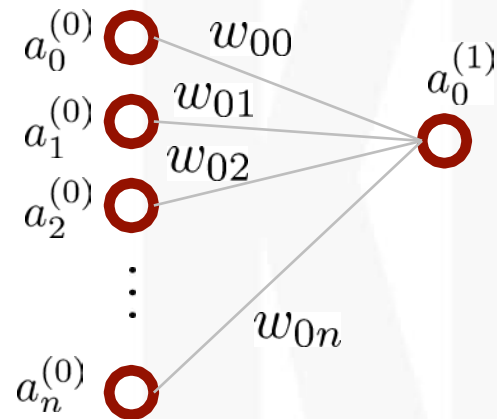
**we are
never
certain...**

Function Behind a Neuron



$$a = \sigma(w_0a_0 + w_1a_1 + \dots + w_na_n + b)$$

Let's Use a Matrix Notation



$$a_0^{(1)} = \sigma \left(w_{00}a_0^{(0)} + w_{01}a_1^{(0)} + \dots + w_{0n}a_n^{(0)} + b_0^{(1)} \right)$$

$$\begin{bmatrix} a_0^{(1)} \\ a_1^{(1)} \\ \vdots \\ a_n^{(1)} \end{bmatrix} = \sigma \left(\begin{bmatrix} w_{00} & w_{01} & \dots & w_{0n} \\ w_{10} & w_{11} & \dots & w_{1n} \\ \vdots & \vdots & \vdots & \vdots \\ w_{k0} & w_{k1} & \dots & w_{kn} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0^{(1)} \\ b_1^{(1)} \\ \vdots \\ b_n^{(1)} \end{bmatrix} \right)$$

Each Layer Can Be Expressed Through Matrix Multiplications

layer 1

layer 0

$$\begin{bmatrix} a_0^{(1)} \\ a_1^{(1)} \\ \vdots \\ a_n^{(1)} \end{bmatrix} = \sigma \left(\begin{bmatrix} w_{00} & w_{01} & \dots & w_{0n} \\ w_{10} & w_{11} & \dots & w_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{k0} & w_{k1} & \dots & w_{kn} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0^{(1)} \\ b_1^{(1)} \\ \vdots \\ b_n^{(1)} \end{bmatrix} \right)$$

$$\mathbf{a}^{(1)} = \sigma \left(\mathbf{W} \mathbf{a}^{(0)} + \mathbf{b}^{(1)} \right)$$

Do It Layer by Layer...

- **input** = layer 0

$$x = a^{(0)}$$

- layer 1

$$\sigma \left(W^{(1)} a^{(0)} + b^{(1)} \right) = a^{(1)}$$

- layer 2

$$\sigma \left(W^{(2)} a^{(1)} + b^{(2)} \right) = a^{(2)}$$

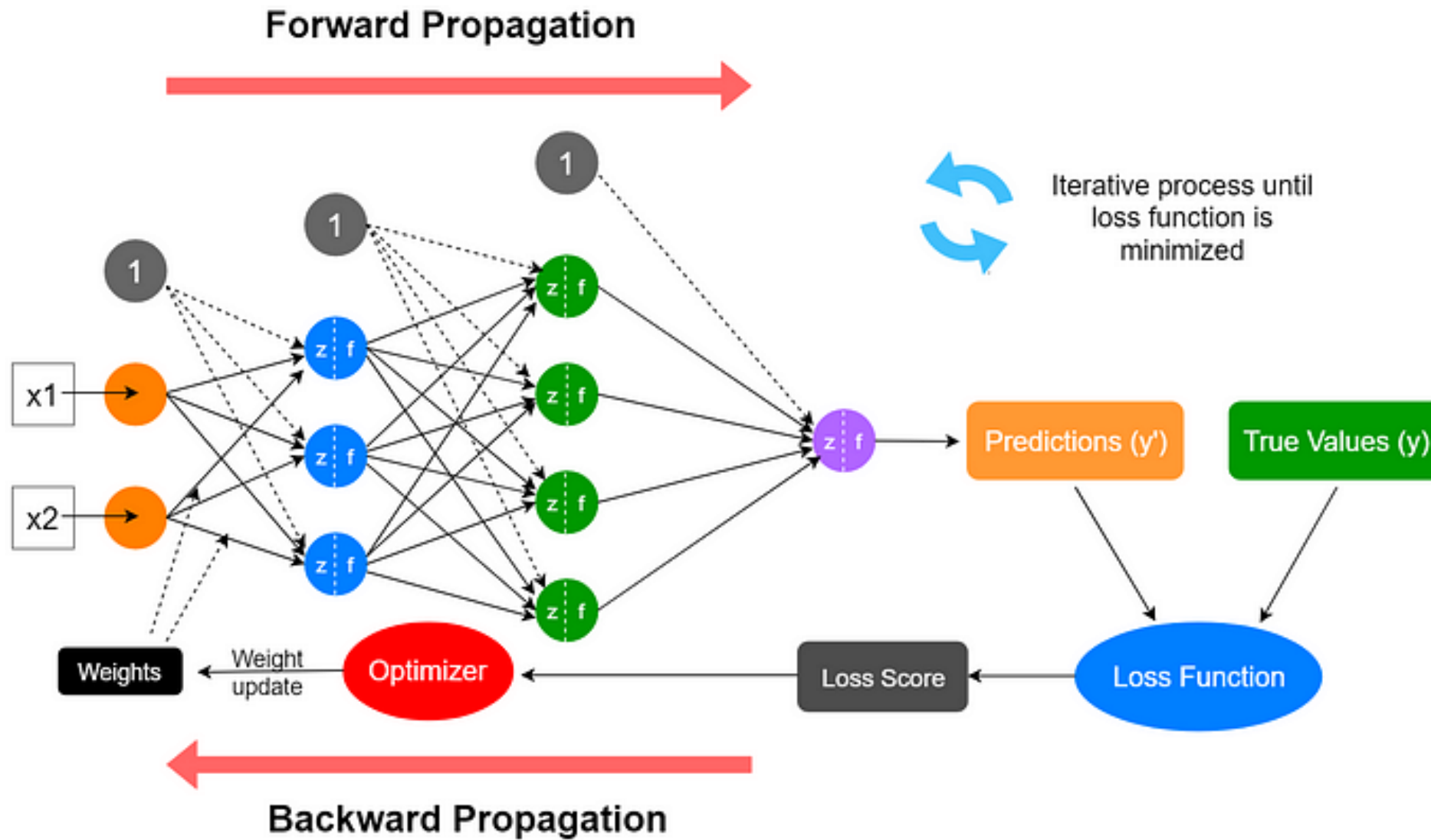
⋮

- layer k = **output**

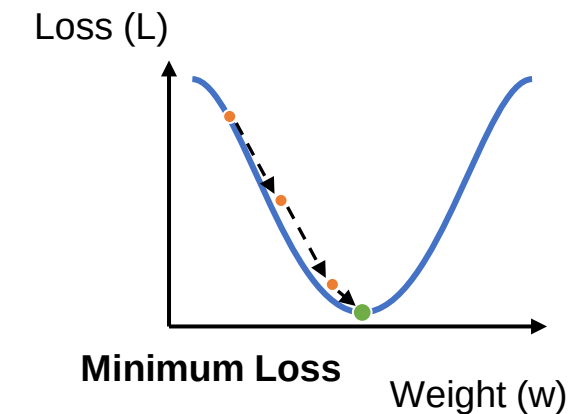
$$\sigma \left(W^{(k)} a^{(k-1)} + b^{(k)} \right) = a^{(k)} = \hat{y}$$

- **That not much more than linear algebra...**

Training perceptrons



Gradient Descent



$$w \leftarrow w - \alpha \cdot \partial L / \partial w$$

Gradient Descent (world's smallest perceptron)

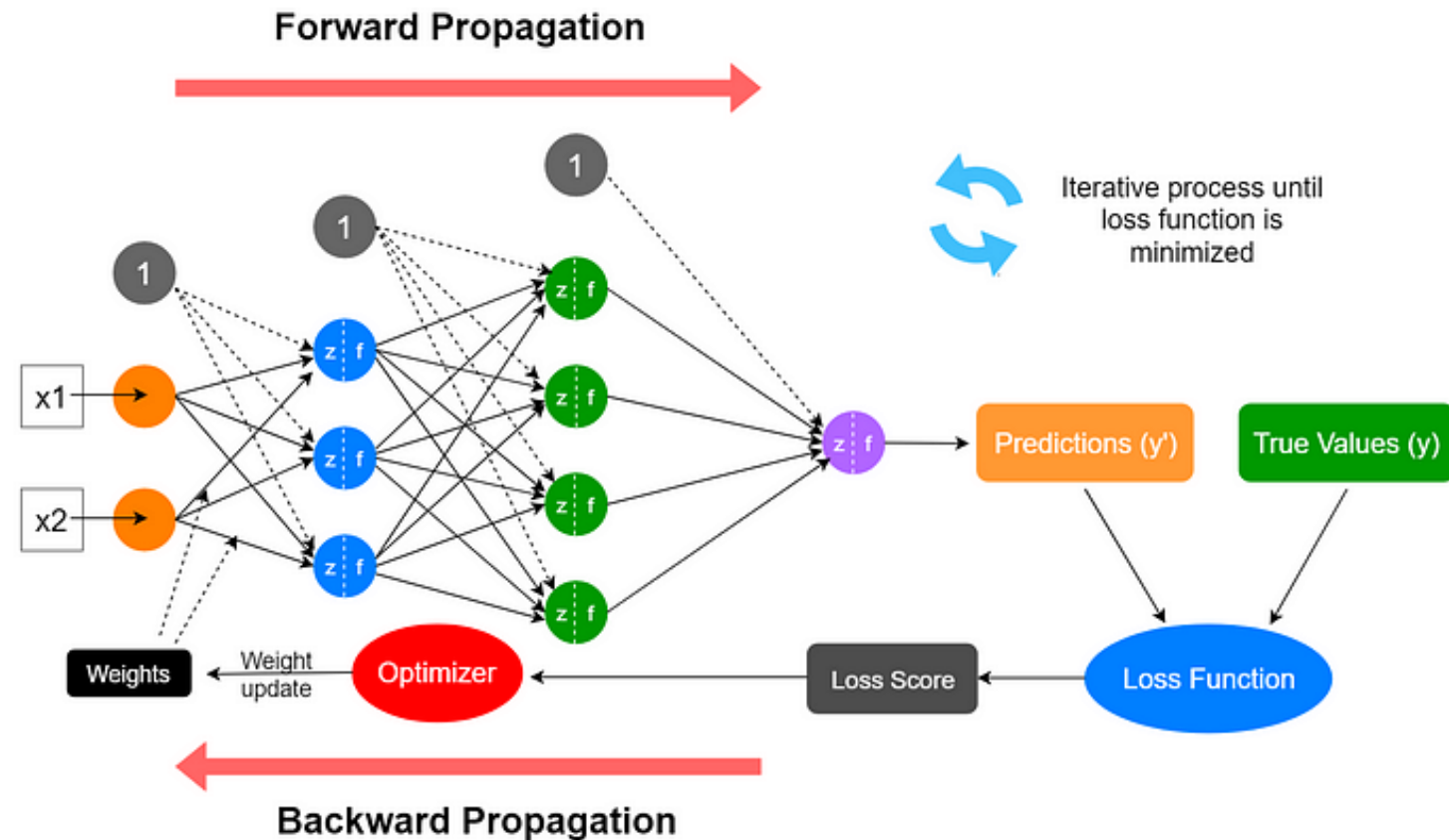
For each sample

1. Predict

- a. Forward pass
- b. Compute Loss

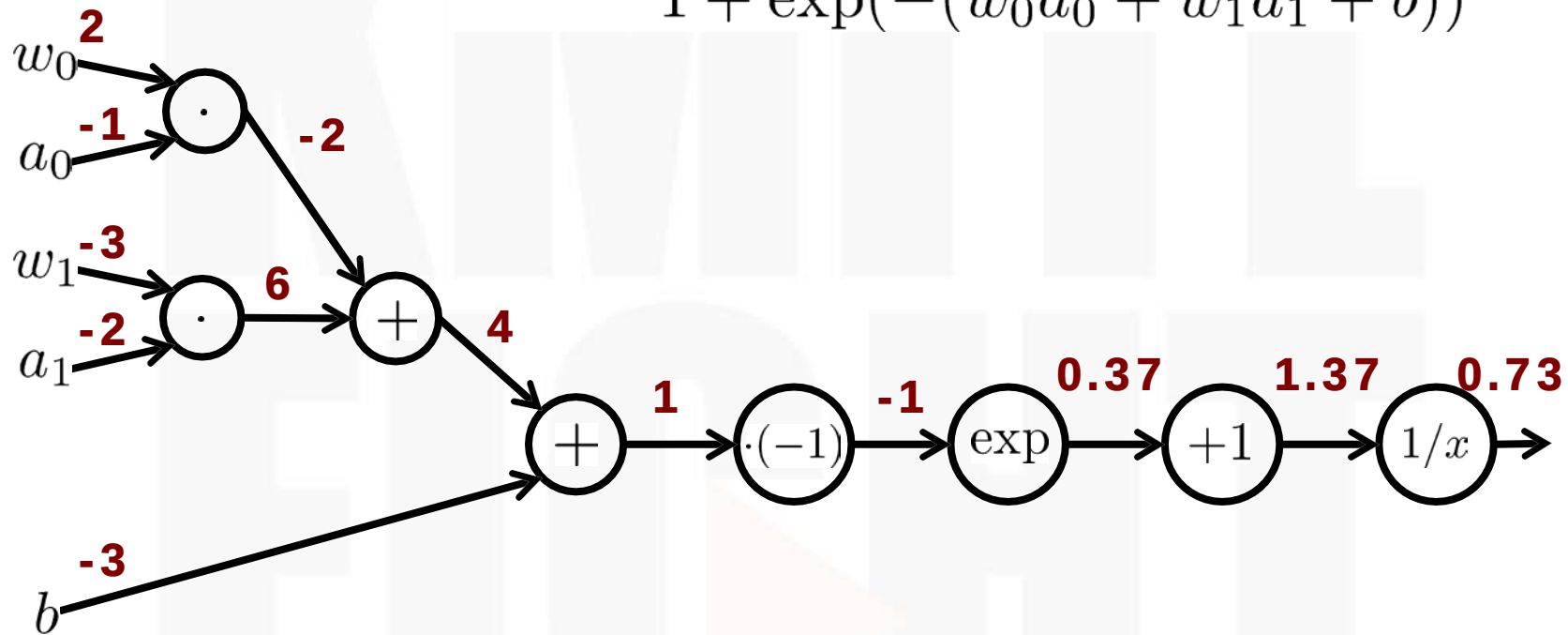
2. Update

- a. Back Propagation
- b. Gradient update

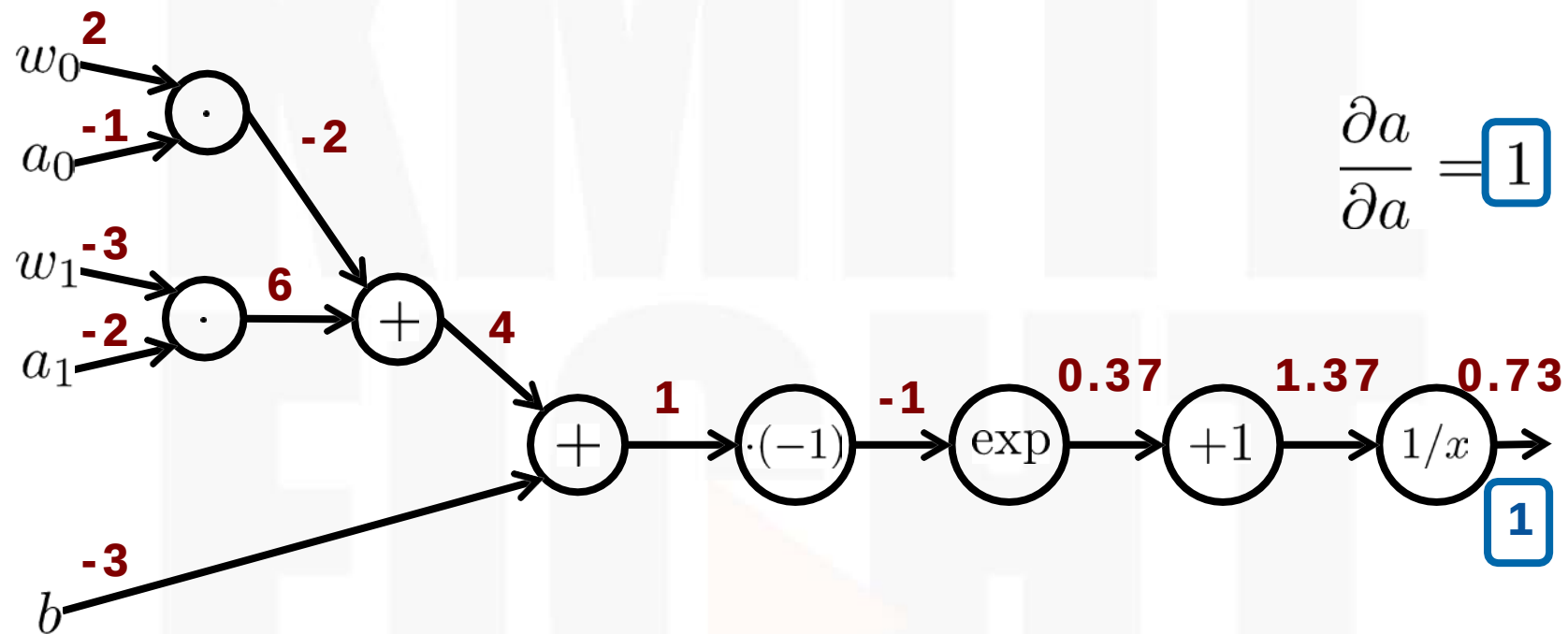


Forward Pass

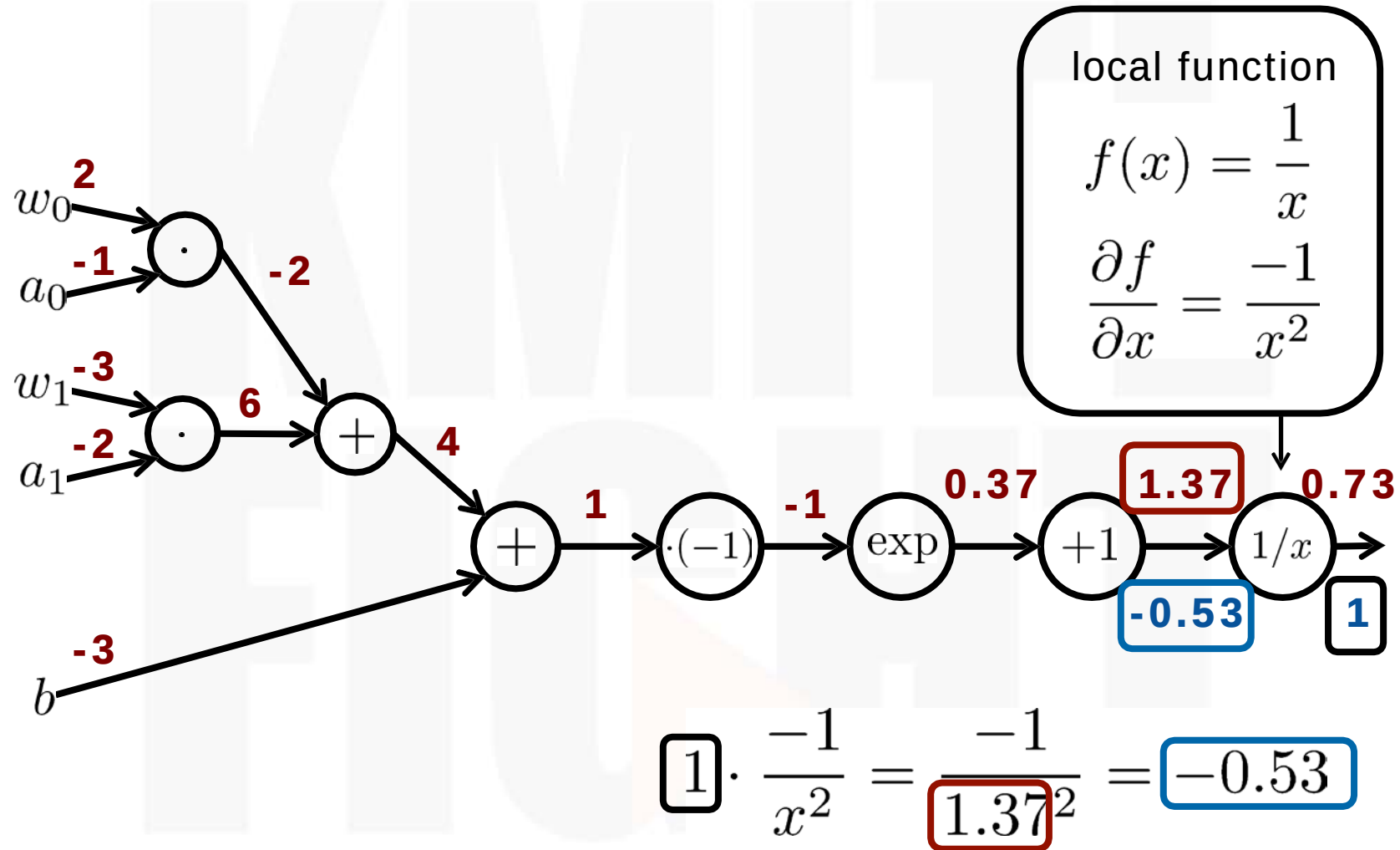
$$a(x) = \frac{1}{1 + \exp(-(w_0 a_0 + w_1 a_1 + b))}$$



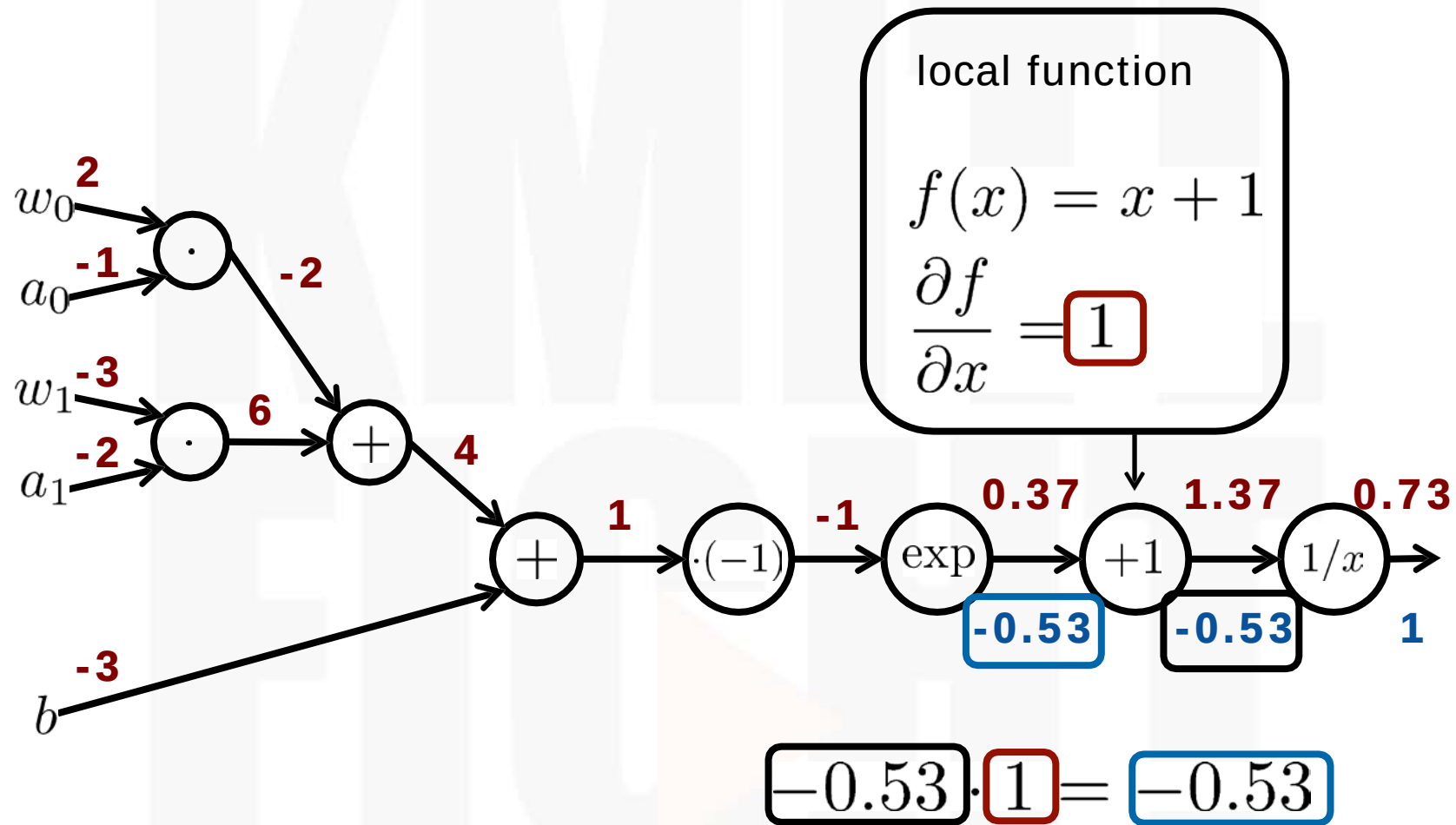
Backward Pass



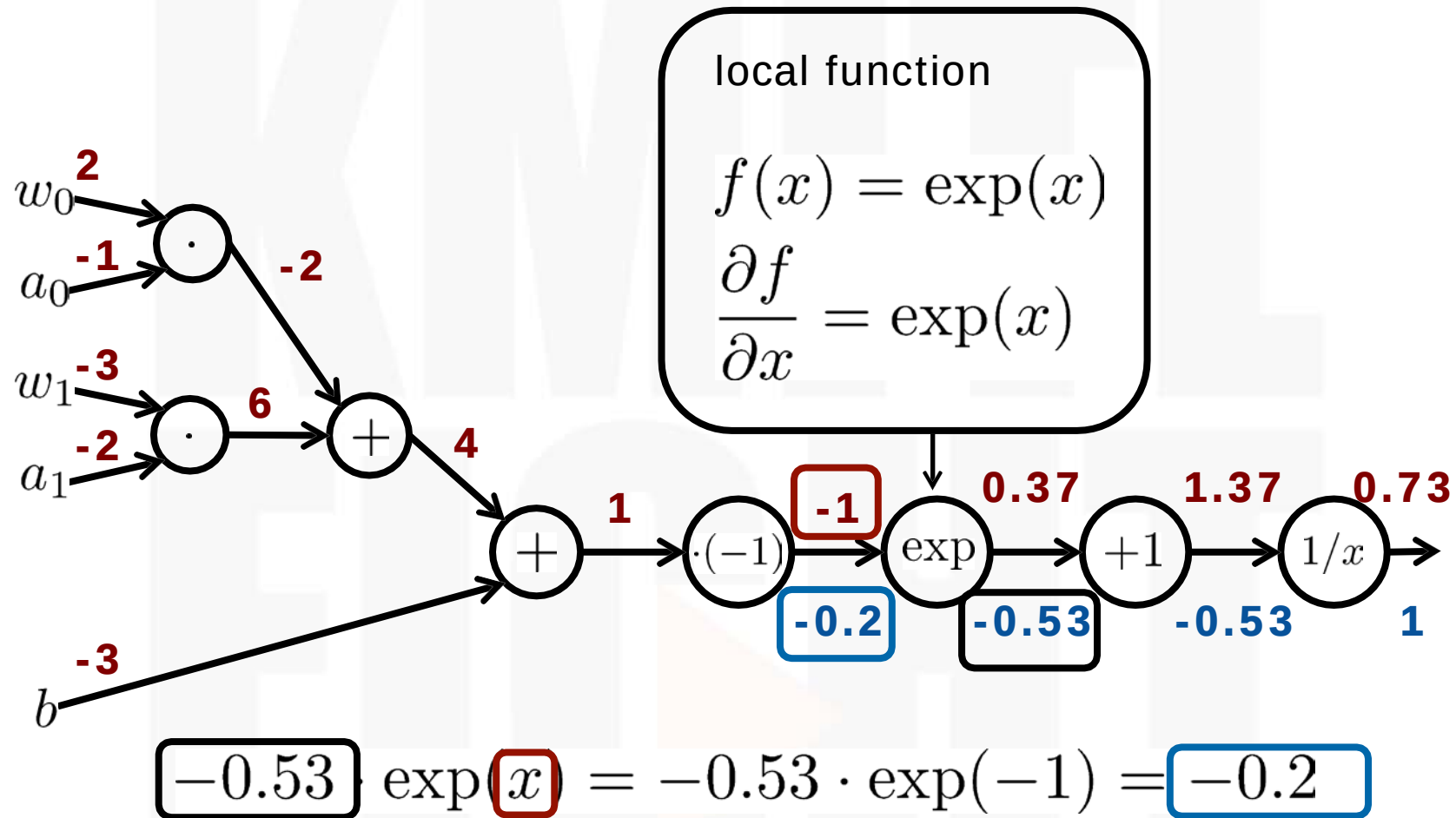
Backward Pass



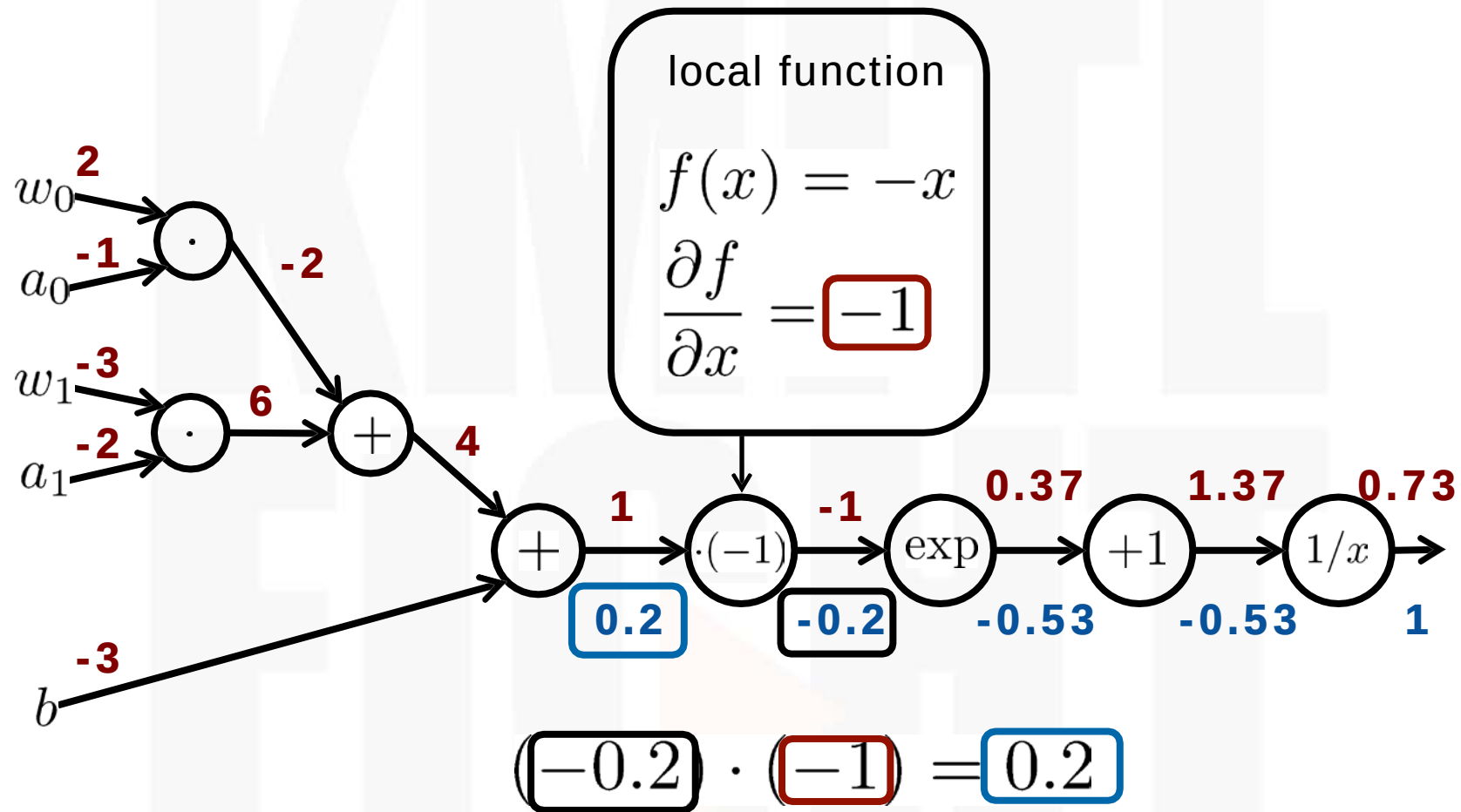
Backward Pass



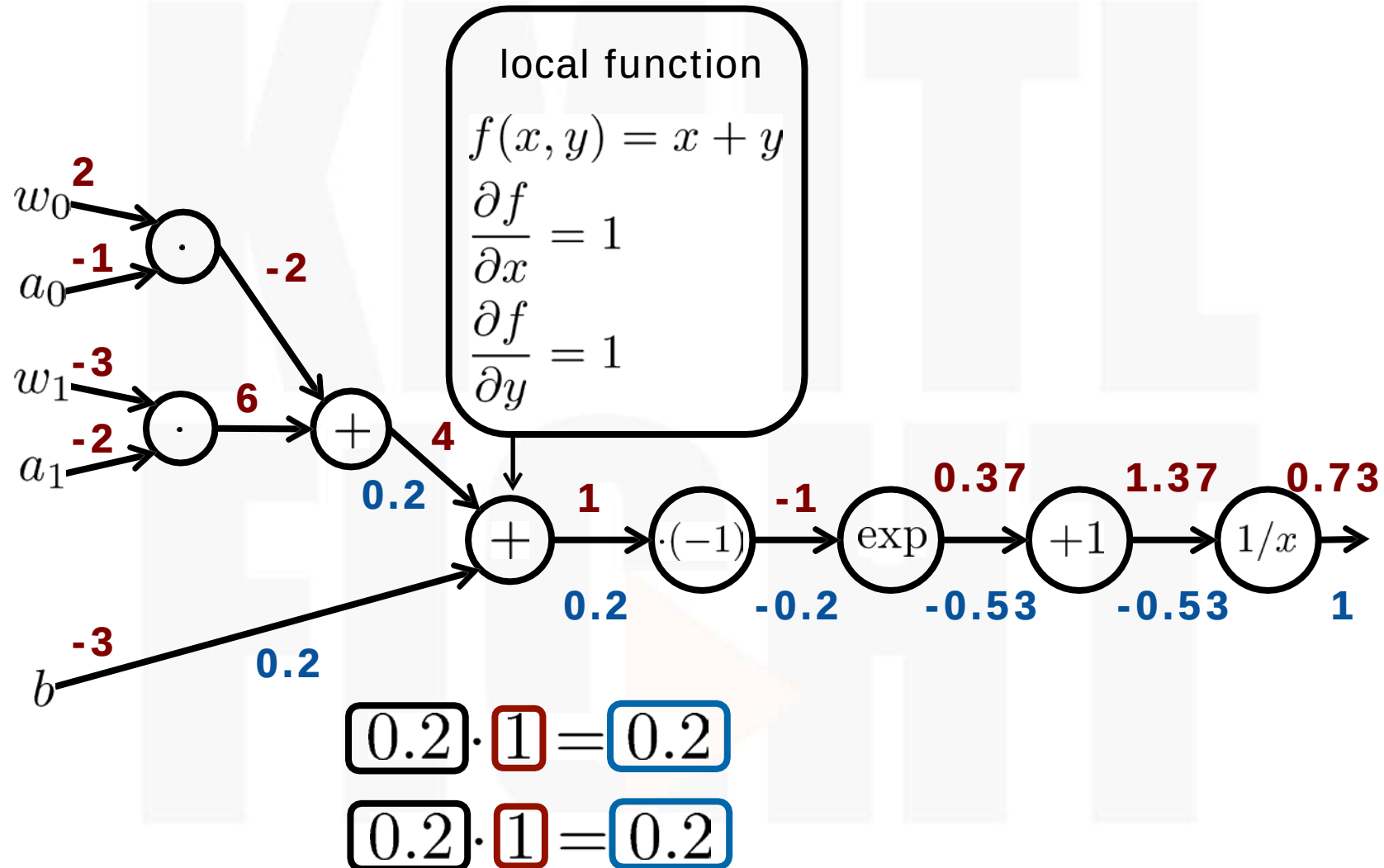
Backward Pass



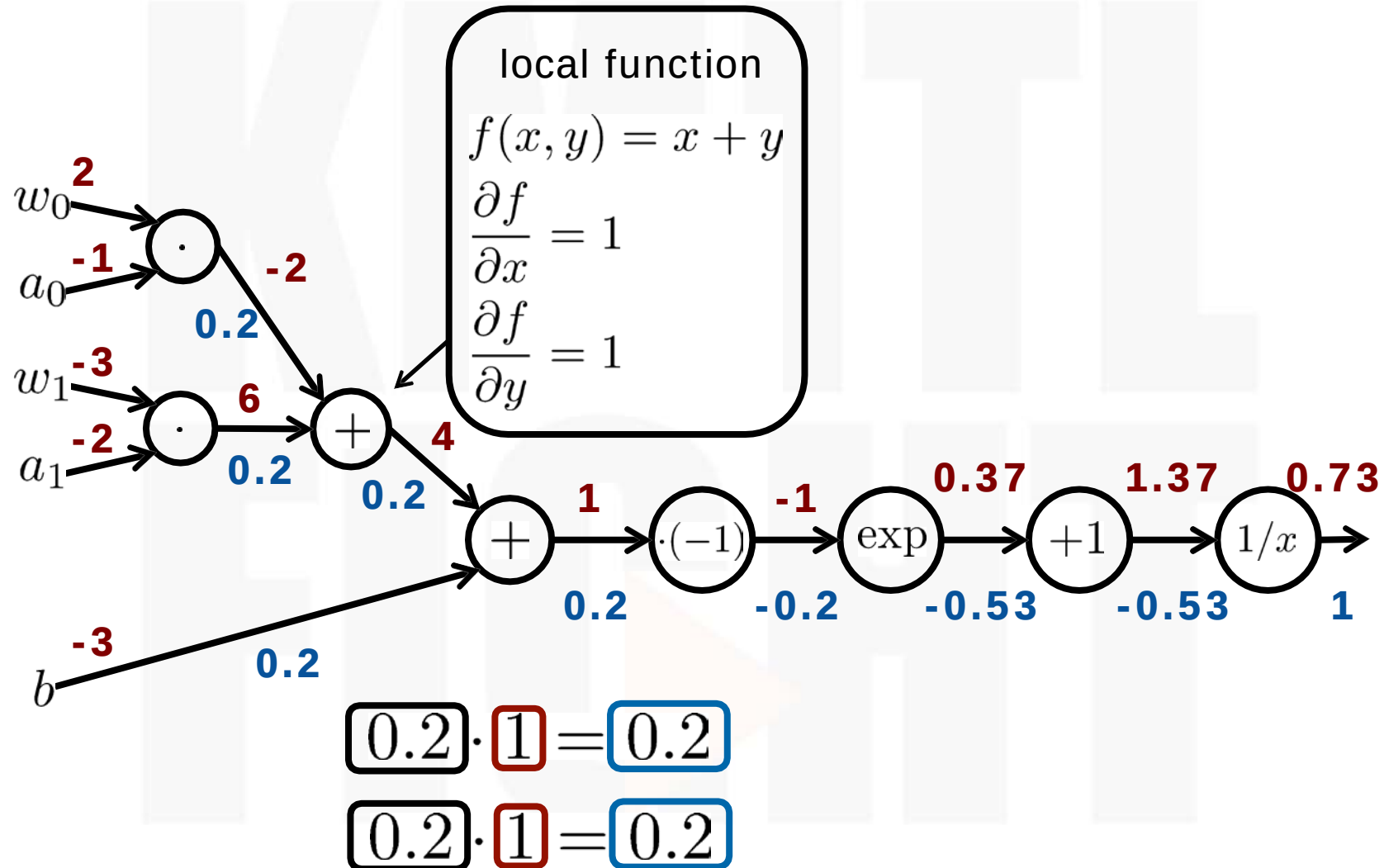
Backward Pass



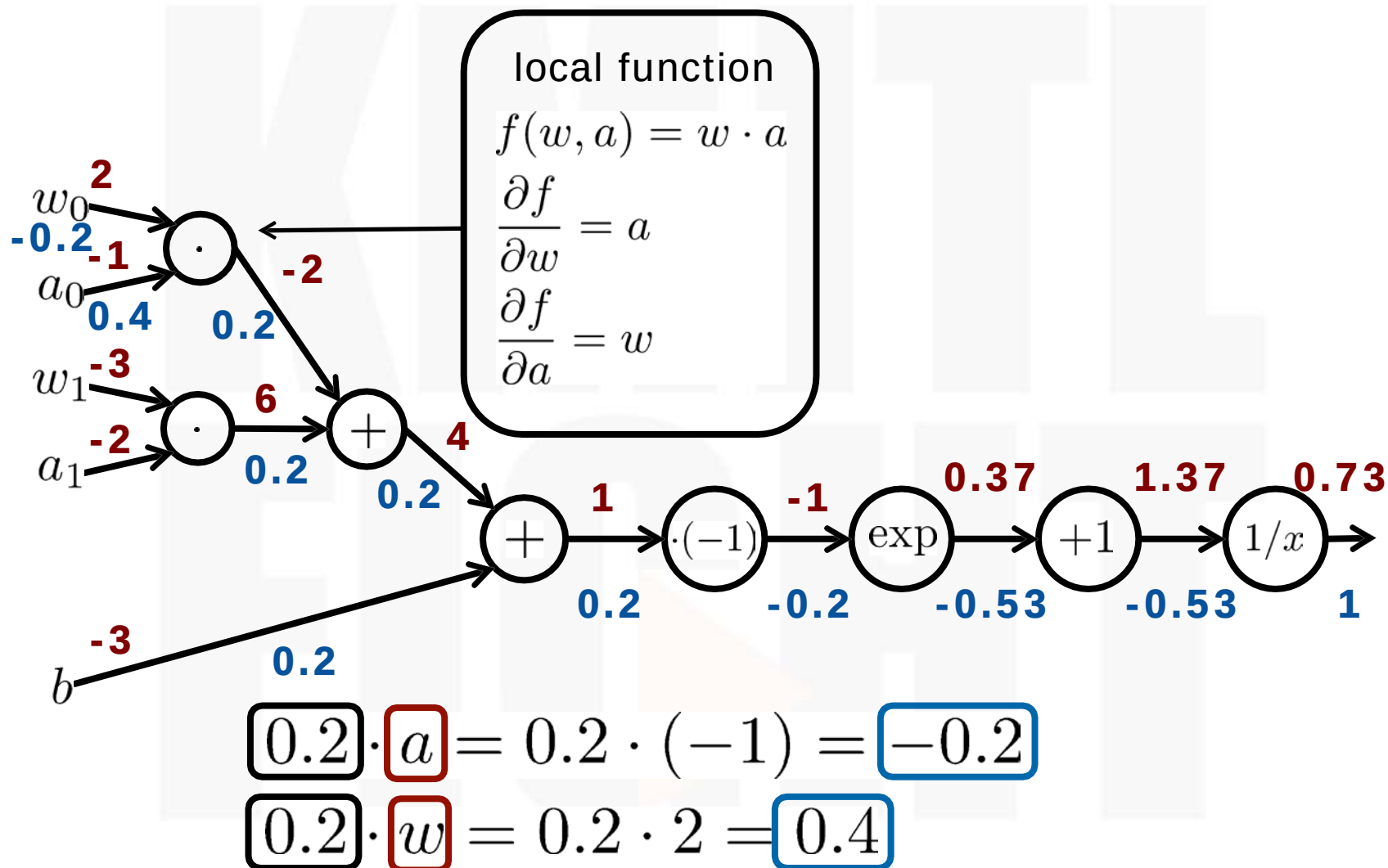
Backward Pass



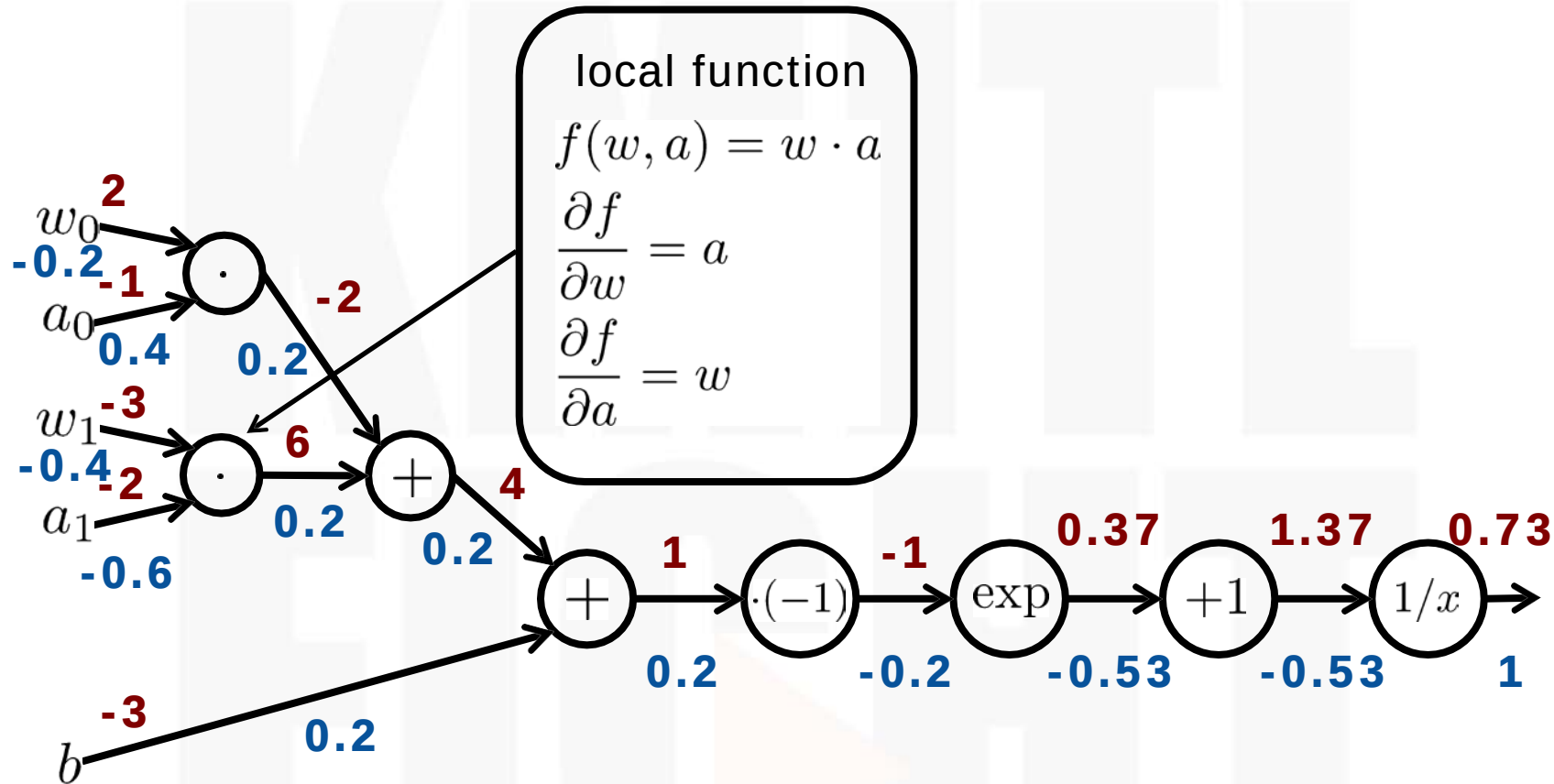
Backward Pass



Backward Pass



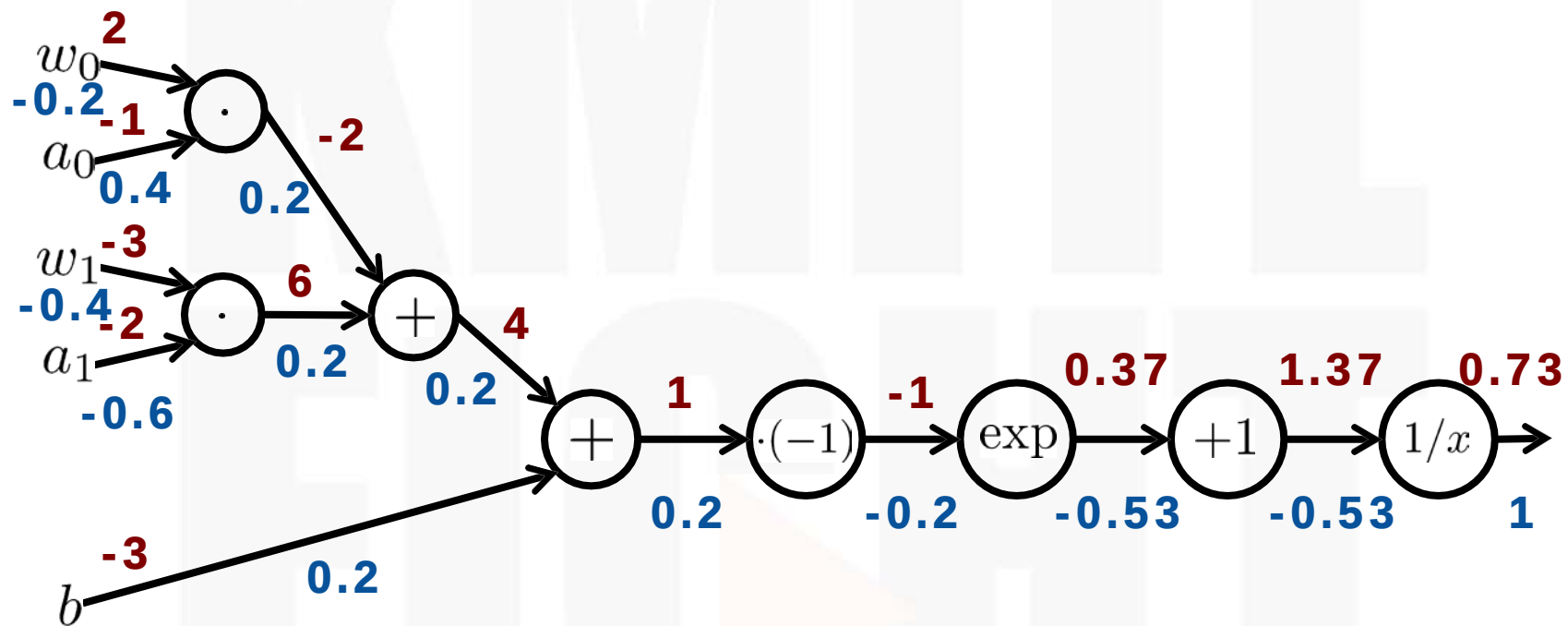
Backward Pass



$$0.2 \cdot a = 0.2 \cdot (-2) = -0.4$$

$$0.2 \cdot w = 0.2 \cdot (-3) = -0.6$$

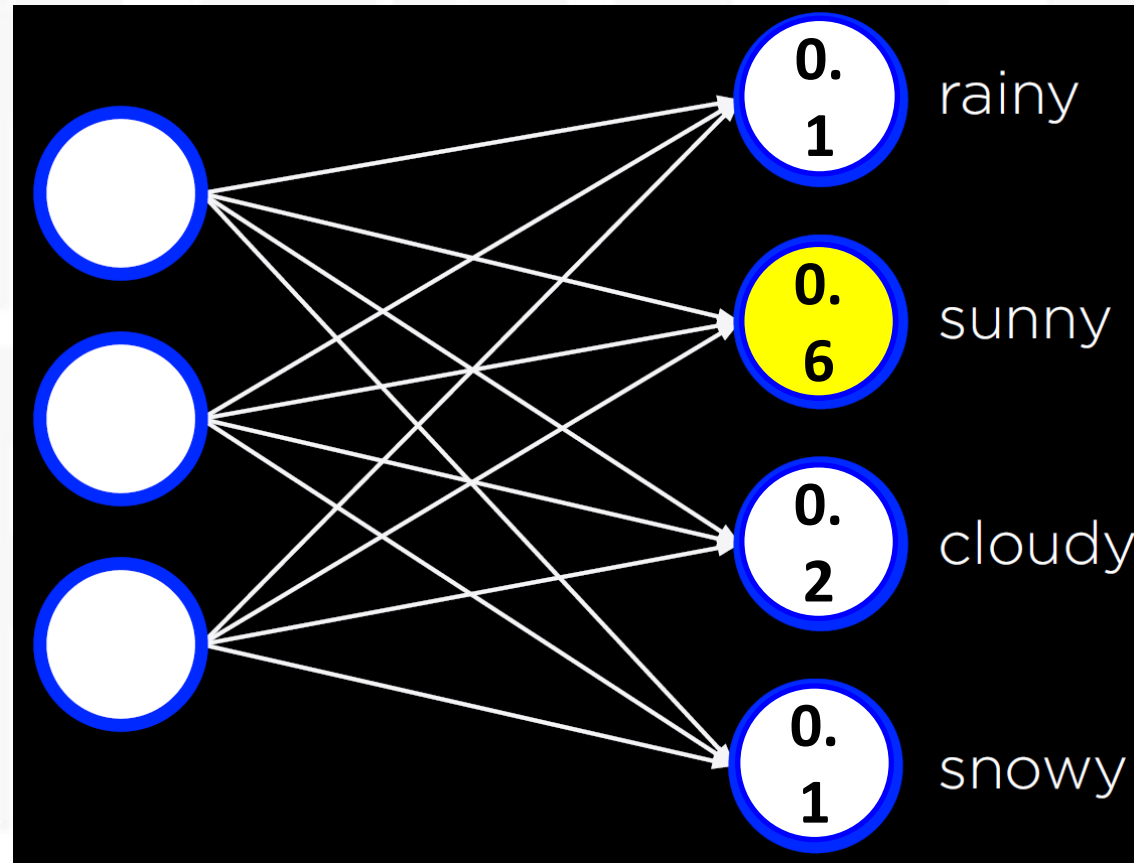
Backward Pass



$$\nabla \sigma|_{[2, -1, -3, -2, -3]^T} = [-0, 2, 0.4, -0.4, -0.6, 0.2]^T$$

- What is Deep learning
- Artificial neural network (ANN)
- Convolutional neural network (CNN)
- Recurrent Neural Network (RNN)
- Generative adversarial network (GAN)

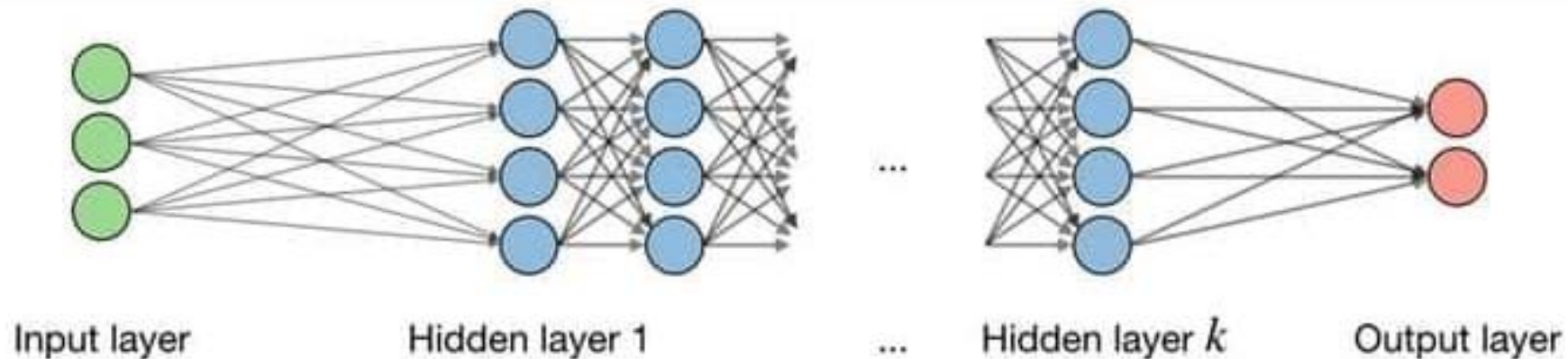
- How then would this neural network work?



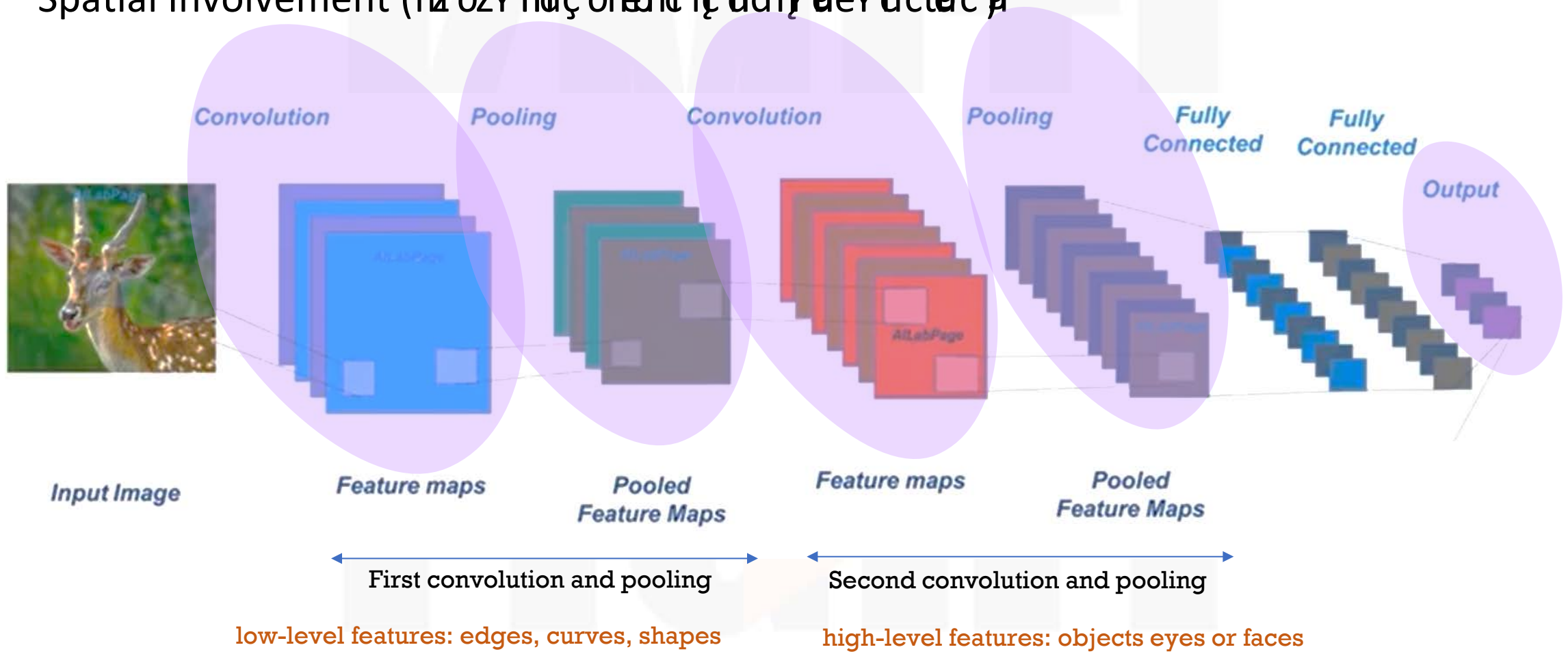
- What is Deep learning
- Artificial neural network (ANN)
- Convolutional neural network (CNN)
- Recurrent Neural Network (RNN)
- Generative adversarial network (GAN)

Introduction to Convolutional Neural Networks (CNNs)

Neural Network Concept



Spatial Involvement (พื้นที่ที่เกี่ยวข้อง)

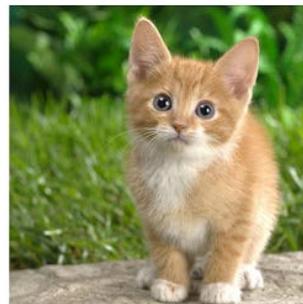


The main types of tasks

Computer vision

- Object Recognition / classification
- Object Detection
- Image Segmentation
- Tracking Object in Videos

Classification



CAT

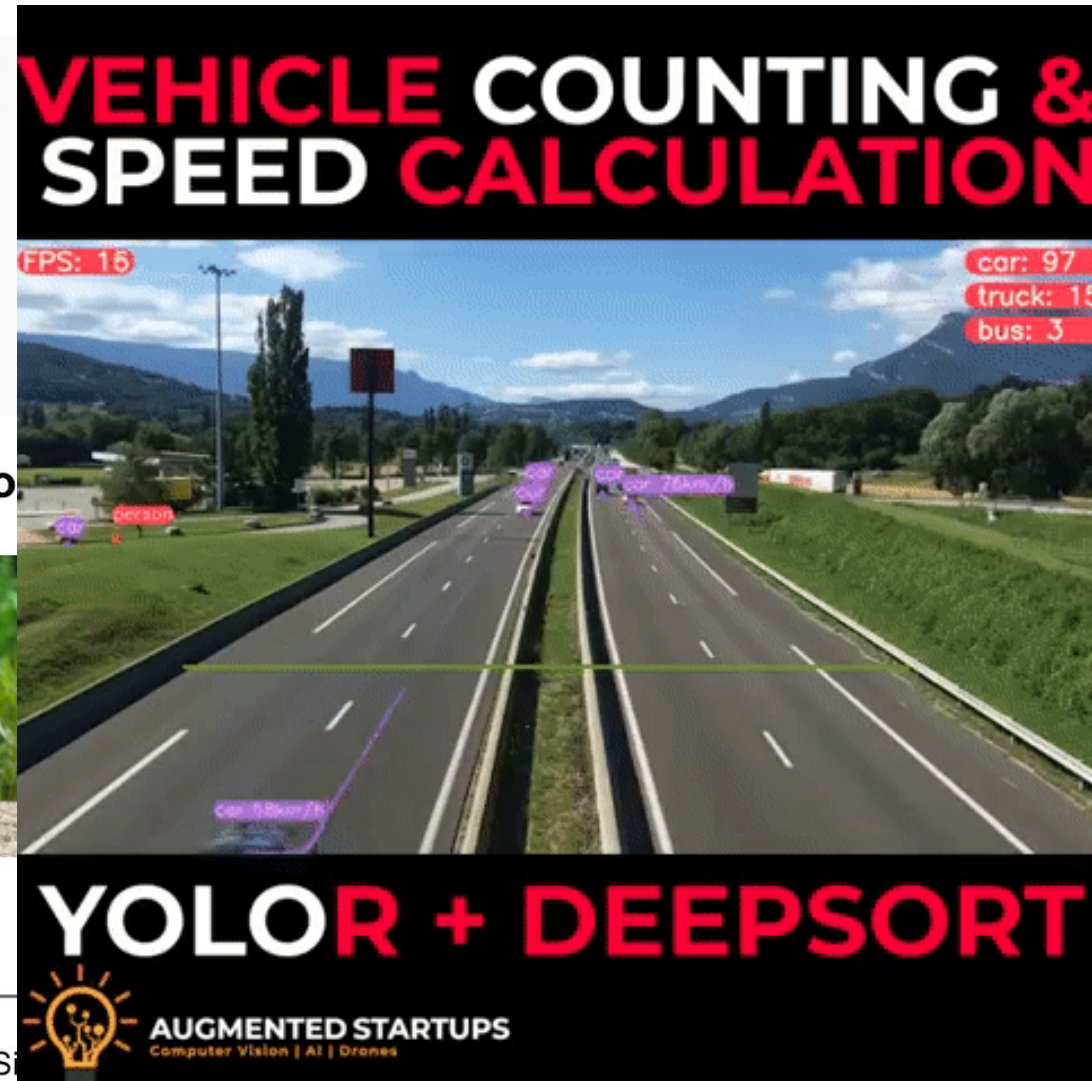


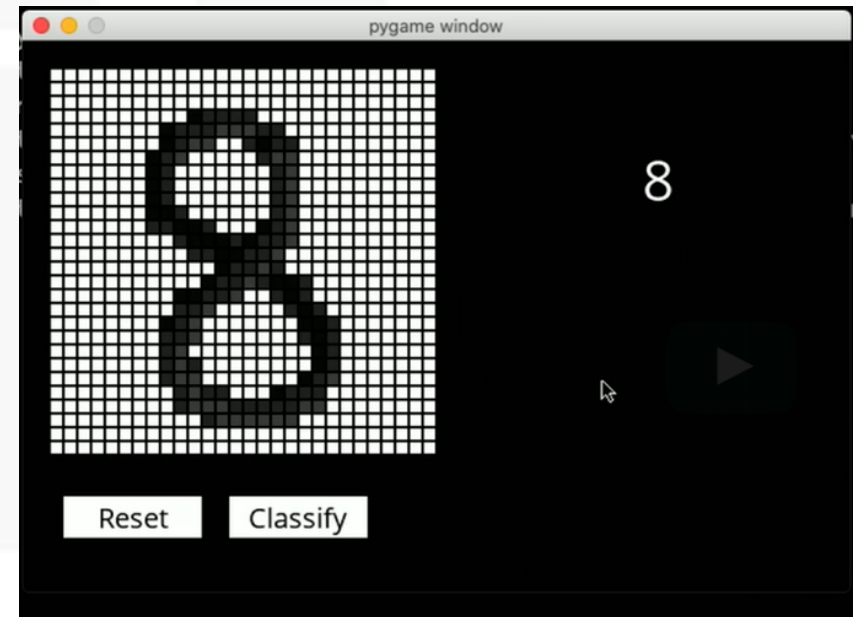
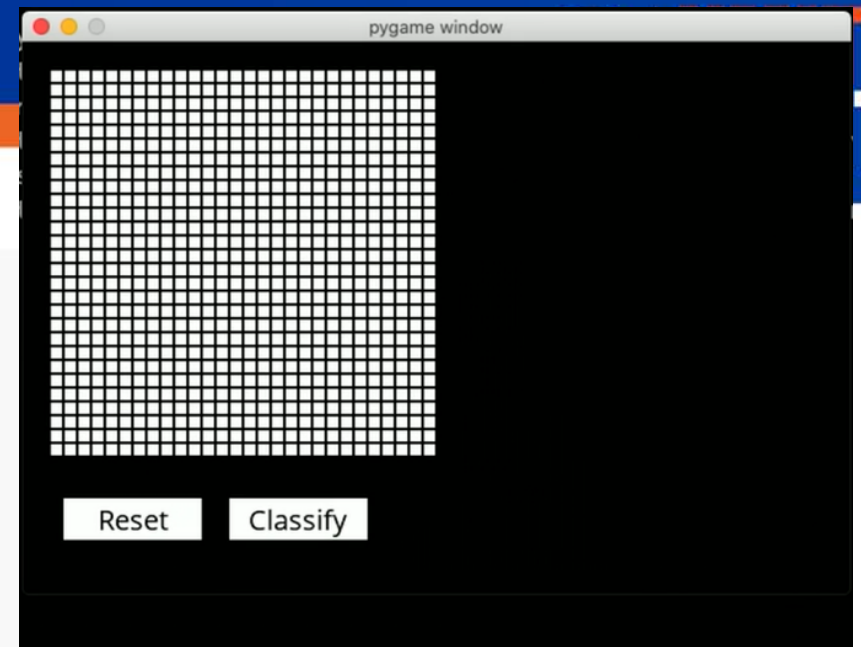
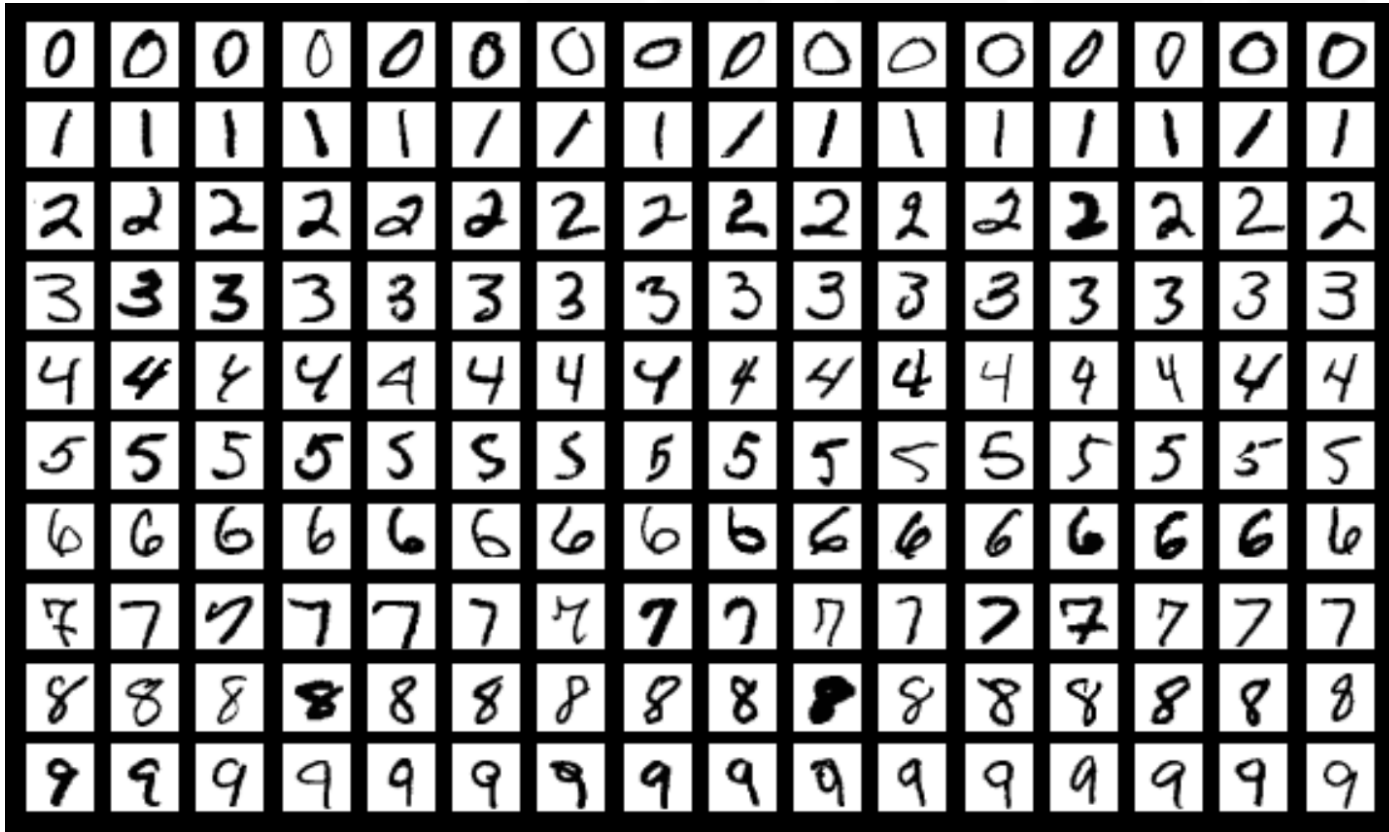
Image Segmentation



DOG, DUCK

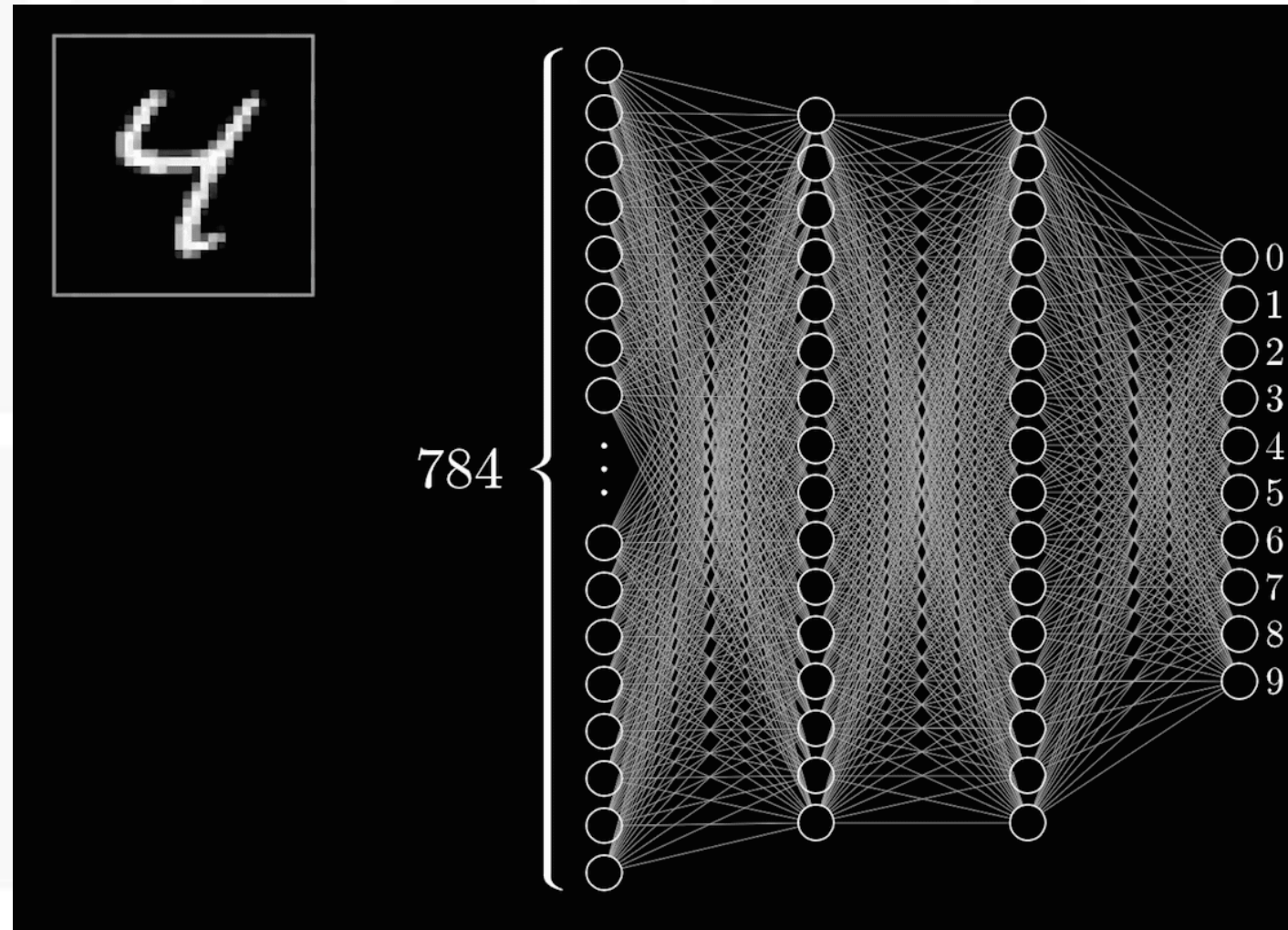
Convolutional neural network (CNN)

MNIST

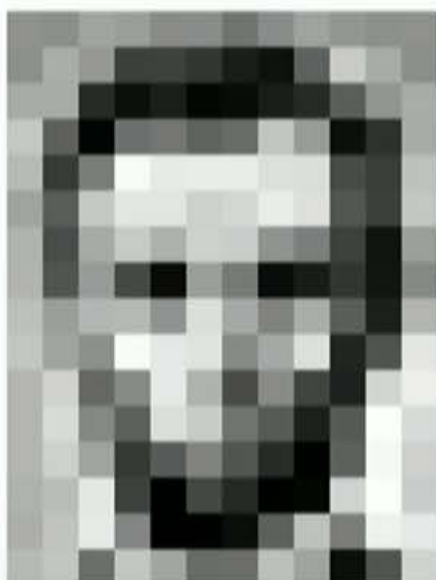


MNIST

สมมติว่าเรามี Neural Network
ที่ใช้ประมวลผลกับข้อมูลภาพขนาด 28×28 px
(784 px^2) จาก MNIST Dataset



Tasks in Computer Vision



Input Image



167	153	174	168	150	162	129	161	172	161	165	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	169	181
206	109	5	124	131	111	120	204	166	16	56	180
194	68	137	261	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	188	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	105	36	190
205	174	155	262	236	231	149	178	228	43	95	234
190	216	116	149	236	187	85	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	216
187	196	235	76	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Pixel Representation



classification

Lincoln

Washington

Jefferson

Obama

$$\begin{bmatrix} 0.8 \\ 0.1 \\ 0.05 \\ 0.05 \end{bmatrix}$$

Regression: output variable takes continuous value

Classification: output variable takes class label. Can produce probability of belonging to a particular class

High Level Feature Detection

Let's identify key features in each image category



Nose,
Eyes,
Mouth



Wheels,
License Plate,
Headlights



Door,
Windows,
Steps

X or X?

?

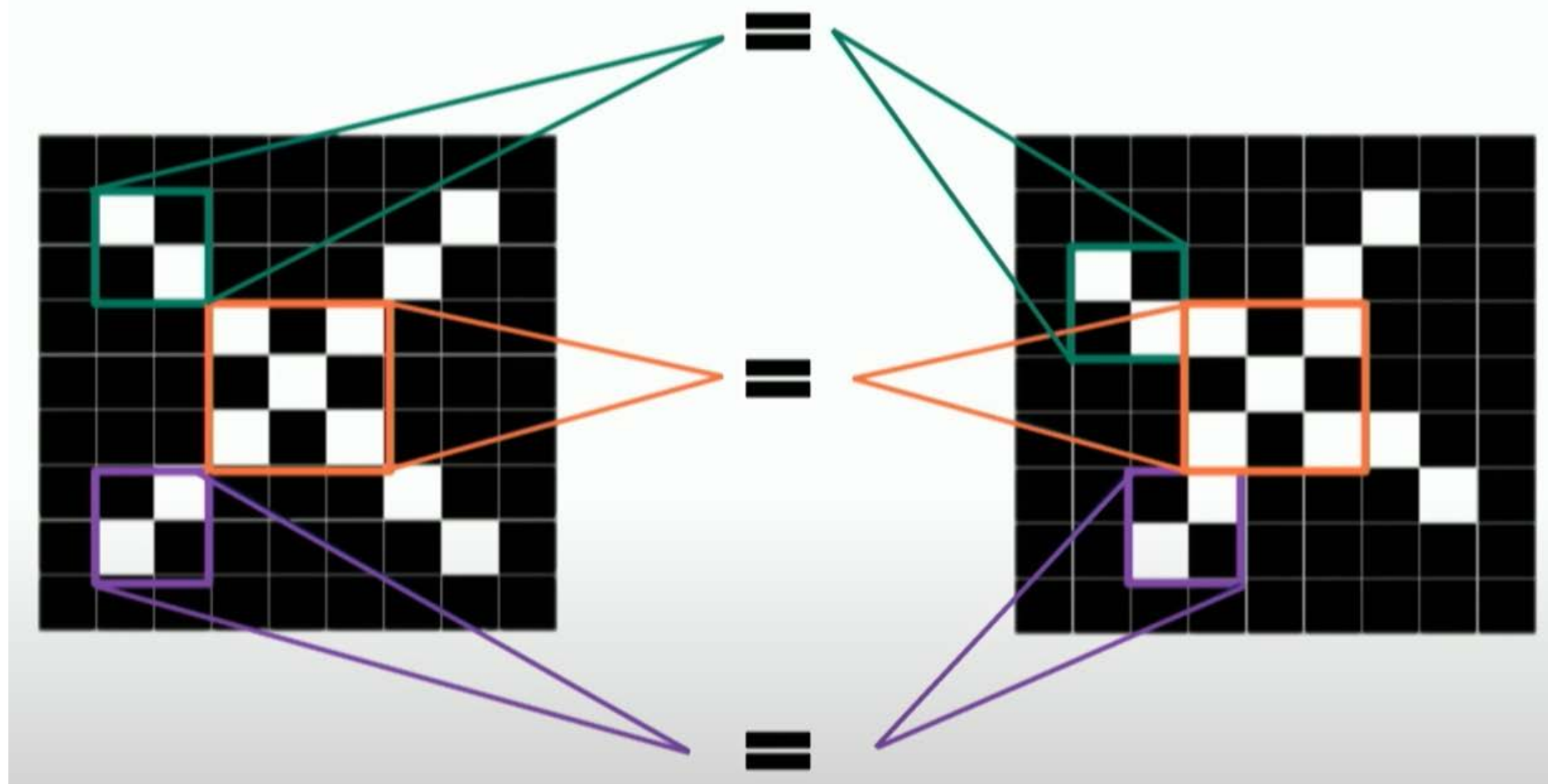
=

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	1	-1	-1	-1
-1	-1	1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	1	-1	-1
-1	-1	-1	1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

Image is represented as matrix of pixel values... and computers are literal!
We want to be able to classify an X as an X even if it's shifted, shrunk, rotated, deformed.

Features of X



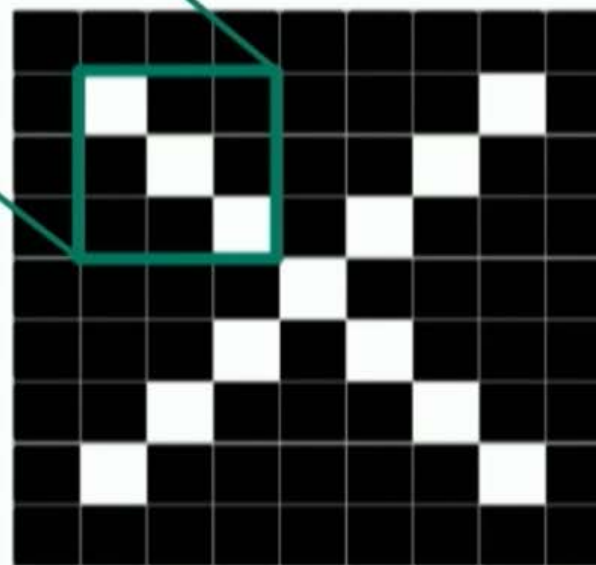
Filters to Detect X Features

filters

1	-1	-1
-1	1	-1
-1	-1	1

1	-1	1
-1	1	-1
1	-1	1

-1	-1	1
-1	1	-1
1	-1	-1



Introduction to Convolutional Neural Networks (CNNs)

Layer Types

There are many types of layers used to build Convolutional Neural Networks, but the ones you are most likely to encounter include:

- Convolutional (CONV)
- Activation (ACT or RELU, where we use the same or the actual activation function)
- Pooling (POOL)
- Fully connected (FC)
- Dropout (DO)

Introduction to Convolutional Neural Networks (CNNs)

The Convolution Operation (Input * Kernel)

Input Slice (6 x 6)

Receptive Field

20	24	11	12	16	19
19	17	20	23	15	9
21	40	25	13	14	8
9	18	8	6	11	22
31	3	7	9	17	23
20	12	3	11	19	30

Sobel Kernel (3 x 3)

1	0	-1
2	0	-2
1	0	-1

*

=

Output (4 x 4)

3			

Sample calculation for first filter location

$$(20 \times 1) + (24 \times 0) + (11 \times -1) + (19 \times 2) + (17 \times 0) + (20 \times -2) + (21 \times 1) + (40 \times 0) + (25 \times -1) = 3$$

20	24	11	12	16	19
19	17	20	23	15	9
21	40	25	13	14	8
9	18	8	6	11	22
31	3	7	9	17	23
20	12	3	11	19	30

*

=

3	27		

Sample calculation for second filter location

$$(24 \times 1) + (11 \times 0) + (12 \times -1) + (17 \times 2) + (20 \times 0) + (23 \times -2) + (40 \times 1) + (25 \times 0) + (13 \times -1) = 27$$

Introduction to Convolutional Neural Networks (CNNs)

Sobel Kernel Example

Sobel Kernel
(3 x 3)

1	0	-1
2	0	-2
1	0	-1

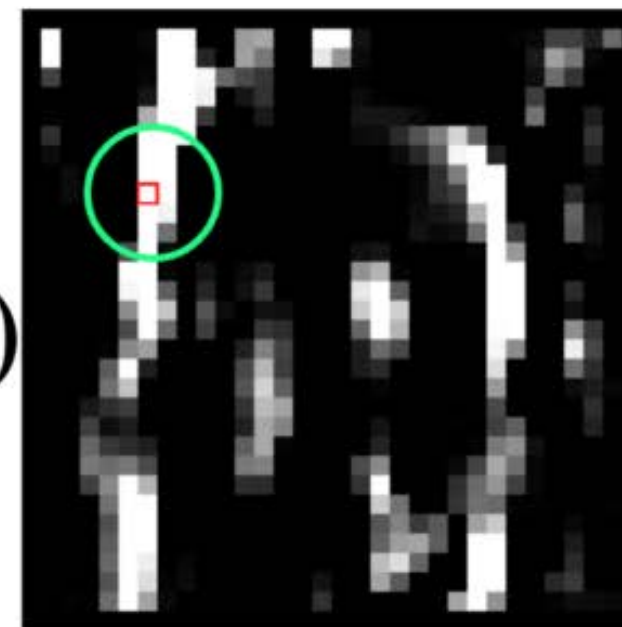


input image

$$\begin{pmatrix} 245 \times 1 + 171 \times 0 + 32 \times -1 \\ + 250 \times 2 + 158 \times 0 + 36 \times -2 \\ + 237 \times 1 + 181 \times 0 + 53 \times -1 \end{pmatrix} = 825$$

kernel:

left sobel



output image

Source: <https://setosa.io/ev/image-kernels/>

Kernel Convolutions

Kernel is an $M \times N$ matrix

3x3 Kernel Example

1	1	1
1	1	1
1	1	1

Kernel Convolution

[illegible]

Diagram illustrating the first step of a 3x3 convolution operation:

- Window:** A 3x3 grid of values, all 247. The center value (247) is highlighted with a red border.
- Kernel:** A 3x3 grid of values, all 1.
- Operation:** The Window is multiplied by the Kernel (indicated by a large 'X' between them).
- Result:** The result is a 3x3 grid of values, all 247 (labeled 'Multiplied').
- Sum:** The sum of the Kernel is 9. The sum of the Multiplied grid is 2223.

New Pixel Value
 $2223 / 9 = \mathbf{247}$

Kernel Convolution

[illegible]

Diagram illustrating the first step of a 3x3 convolution operation:

- Window:** A 3x3 grid of values, all 247. The center value (247) is highlighted with a red border.
- Kernel:** A 3x3 grid of values, all 1.
- Operation:** The Window is multiplied by the Kernel (indicated by a large 'X' between them).
- Multiplied:** The result of the multiplication is a 3x3 grid of values, all 247.
- Sum = 9:** The sum of the Kernel values (1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1).
- Sum = 2223:** The sum of the Multiplied values (247 + 247 + 247 + 247 + 247 + 247 + 247 + 247 + 247).

New Pixel Value
 $2223 / 9 = \mathbf{247}$

Kernel Convolution

[illegible]

Window

247	247	247
247	247	247
247	247	247

X

Kernel

1	1	1
1	1	1
1	1	1

=

Multiplied

247	247	247
247	247	247
247	247	247

Sum = 9

Sum = 2223

New Pixel Value
 $2223 / 9 = \mathbf{247}$

Kernel Convolution

[illegible]

Window

247	247	120
247	247	83
247	247	83

X

Kernel

1	1	1
1	1	1
1	1	1

=

Multiplied

247	247	120
247	247	83
247	247	83

Sum = 9

Sum = 1768

New Pixel Value

$$1768 / 9 =$$

196.4

196

Kernel Convolution

[illegible]

Window

247	120	95
247	83	83
247	83	83

Kernel

1	1	1
1	1	1
1	1	1

Multiplied

247	120	95
247	83	83
247	83	83

Sum = 9

Sum = 1288

New Pixel Value

$$1288 / 9 =$$

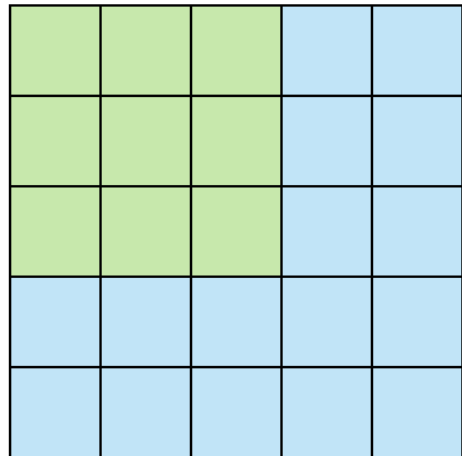
143.1

143

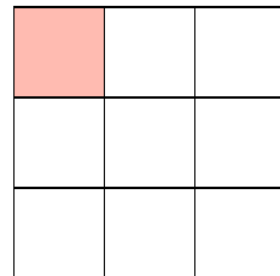
Introduction to Convolutional Neural Networks (CNNs)

Stride and Padding

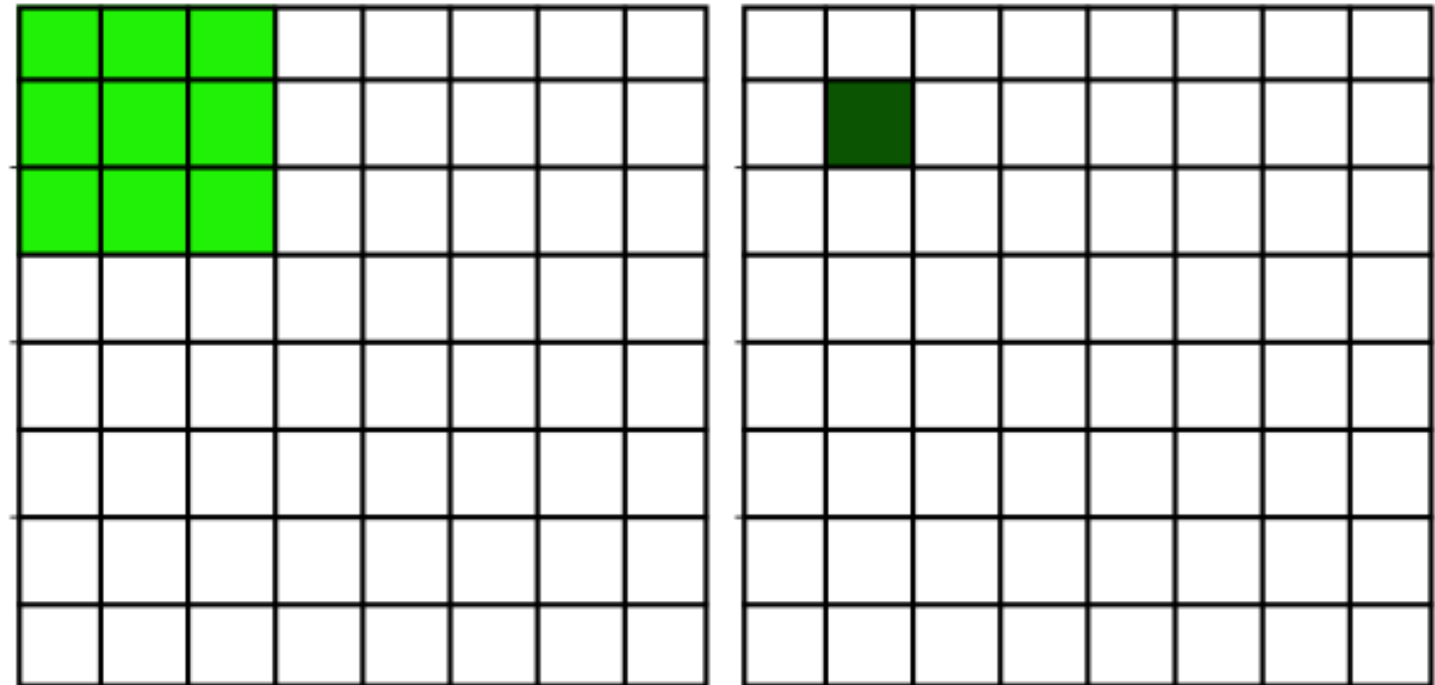
Stride denotes how many steps we are moving in each steps in convolution. By default it is one.



Stride 1

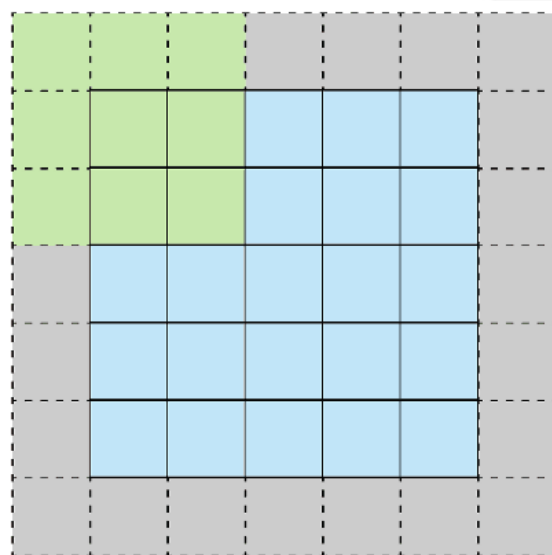


Feature Map

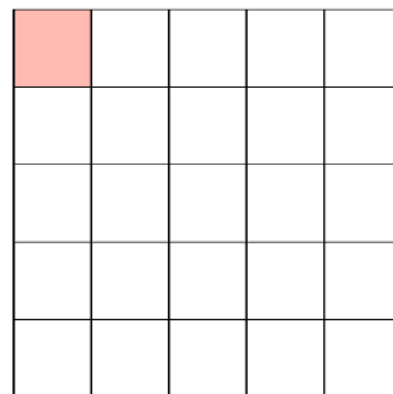


Introduction to Convolutional Neural Networks (CNNs)

We can observe that the size of output is smaller than input. To maintain the dimension of output as in input, we use padding. Padding is a process of adding zeros to the input matrix symmetrically. In the following example, the extra grey blocks denote the padding. It is used to make the dimension of output same as input.



Stride 1 with Padding



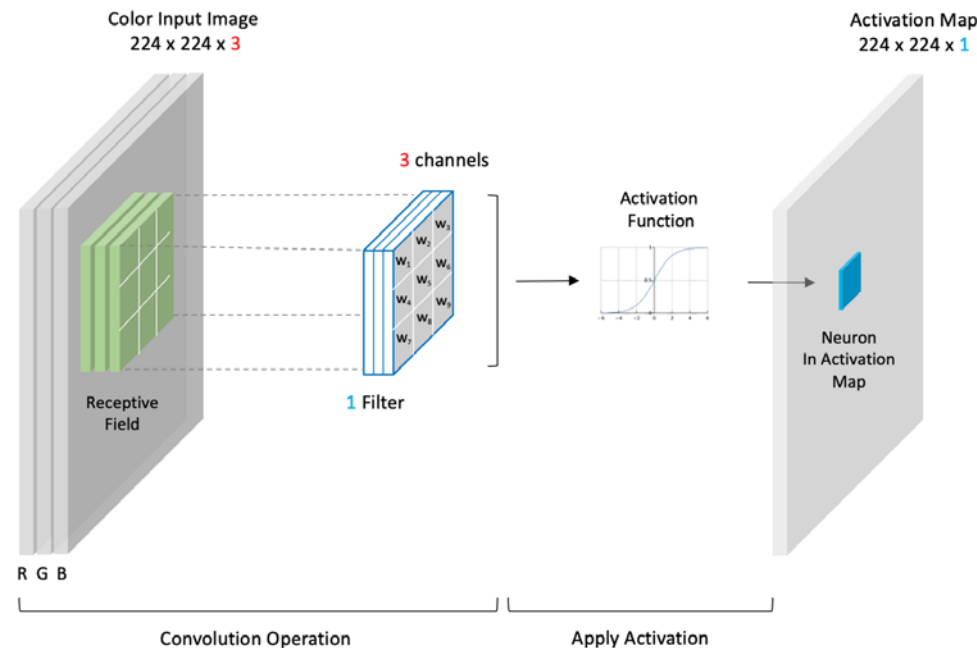
Feature Map

Introduction to Convolutional Neural Networks (CNNs)

Convolutional Layers

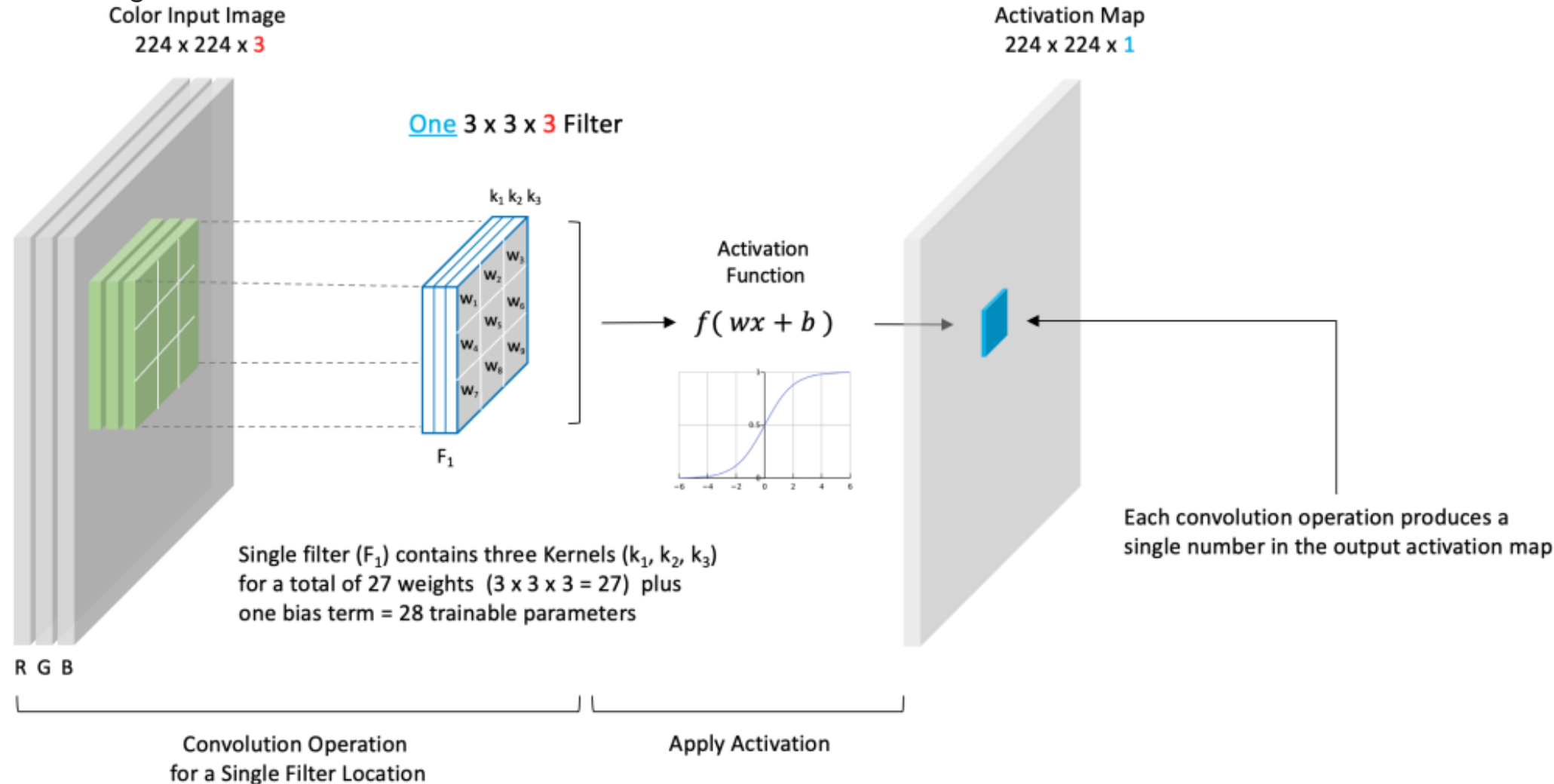
A convolutional layer can be thought of as the “eyes” of a CNN. The neurons in a convolutional layer look for specific features

The input to the very first convolutional layer is the input image. The input image is typically either a grayscale image (single channel) or a color image (3 channels).



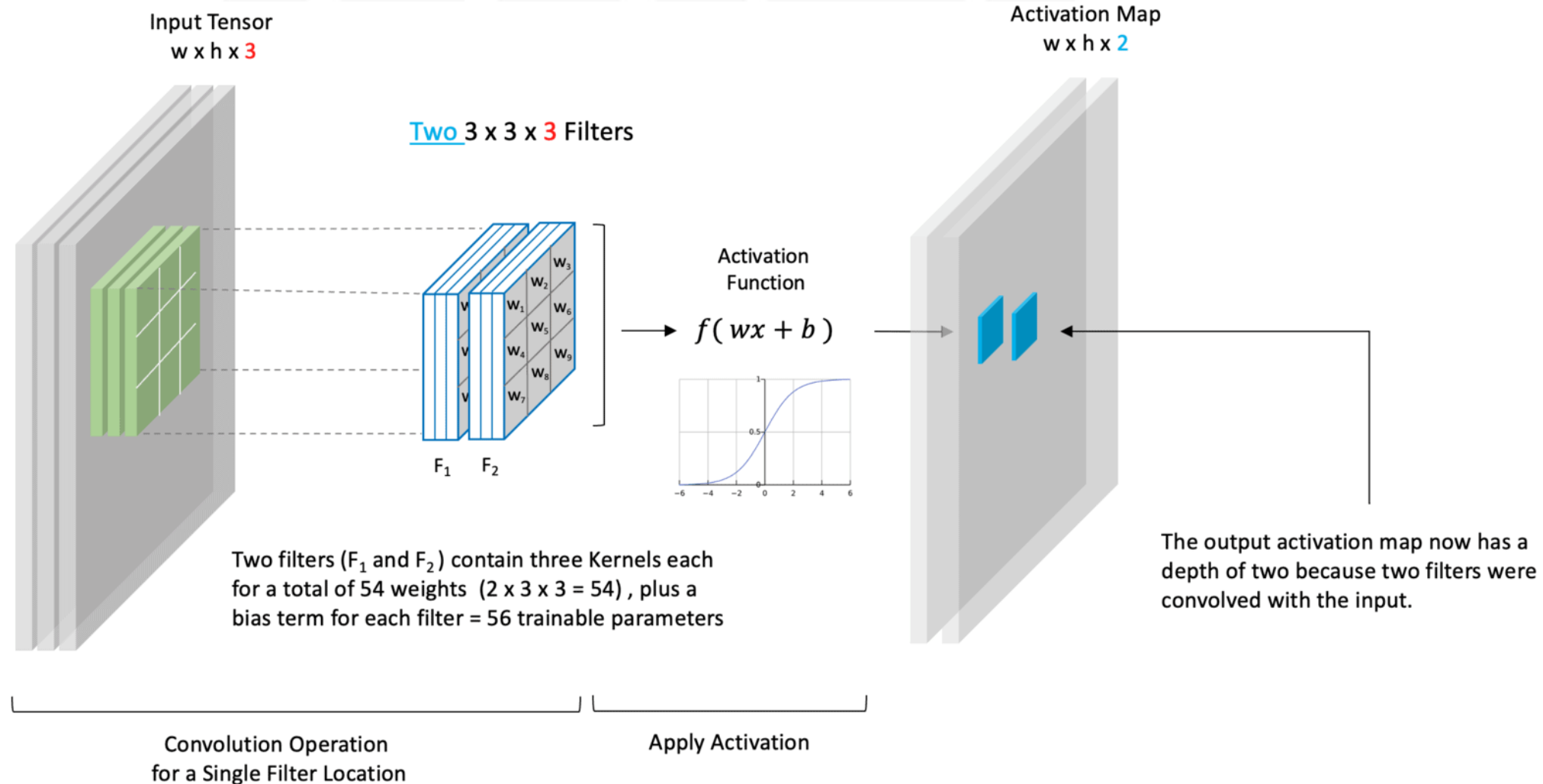
Introduction to Convolutional Neural Networks (CNNs)

Convolutional Layer with a Single Filter



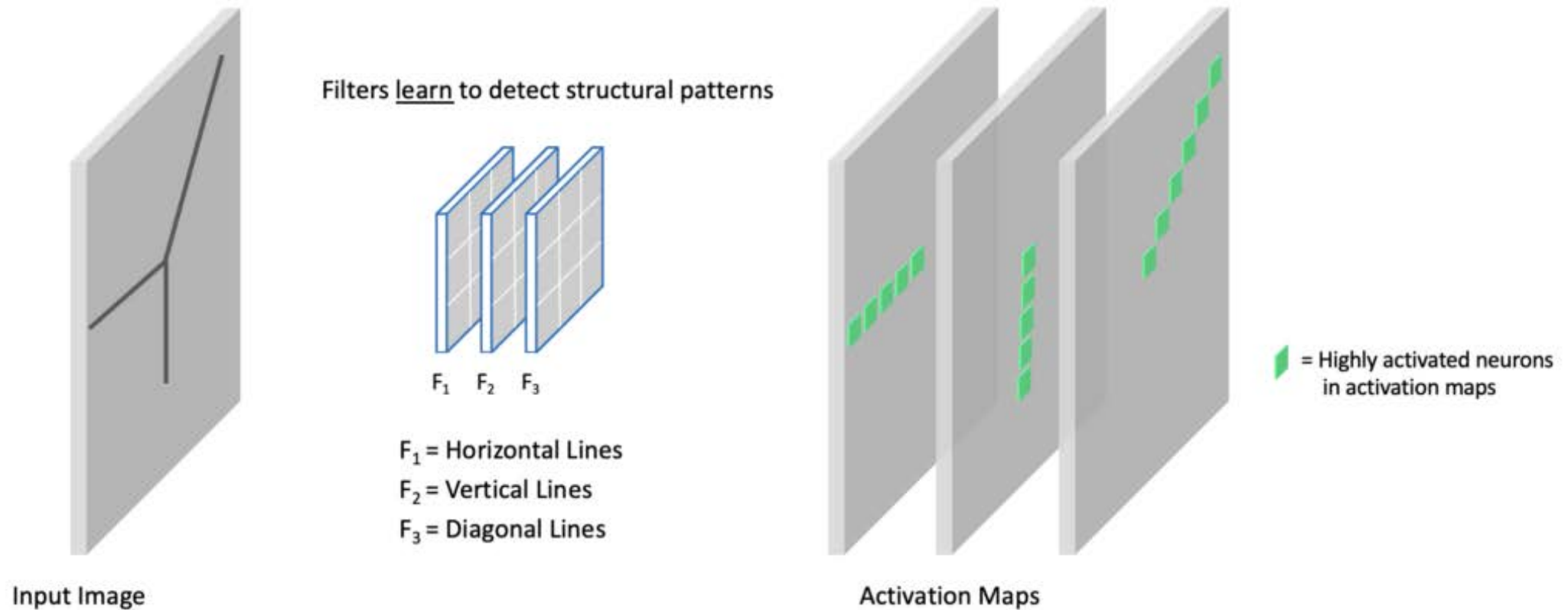
Introduction to Convolutional Neural Networks (CNNs)

Convolutional Layer with a Two Filters



Introduction to Convolutional Neural Networks (CNNs)

Filters Learn to Detect Structure



Sharpen

Kernel

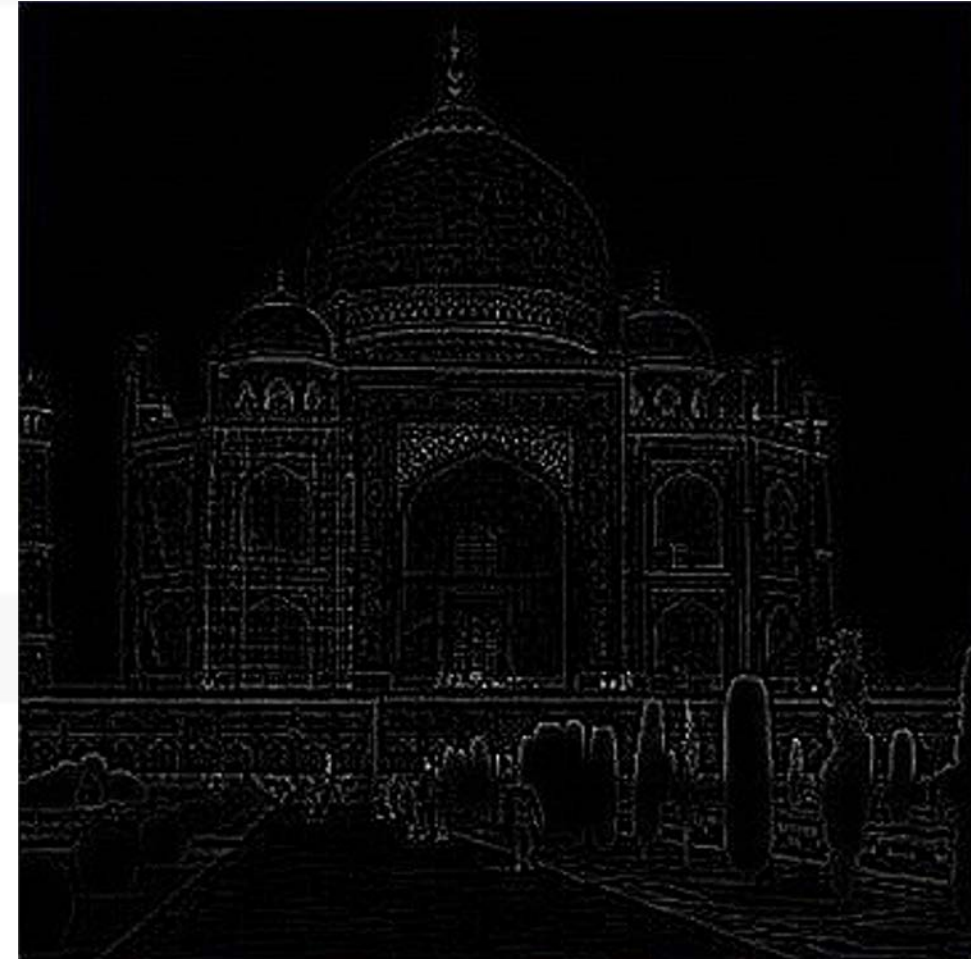
0	-1	0
-1	5	-1
0	-1	0



Kernel Convolutions

Edge
Highlight
Kernel

0	1	0
1	-4	1
0	1	0



Emboss

Kernel

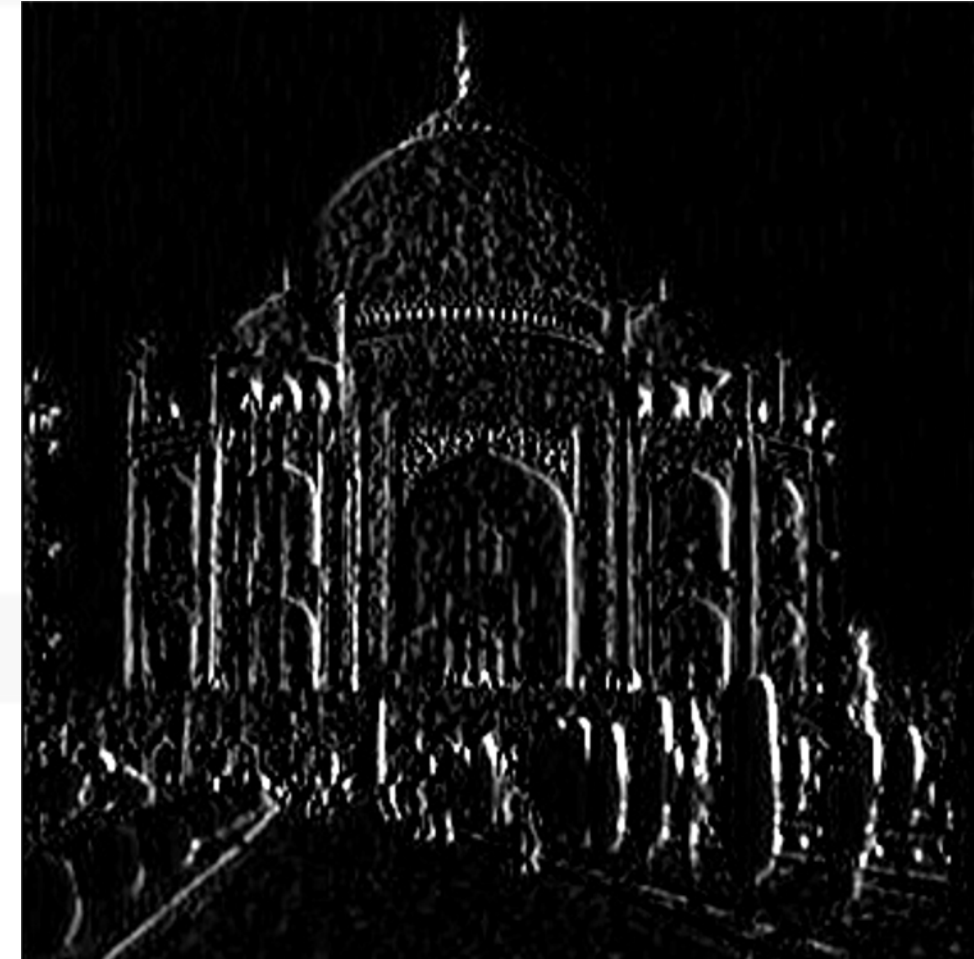
-2	-1	0
-1	1	1
0	1	2



Horizontal "Derivative"

Kernel

1	0	-1
2	0	-2
1	0	-1



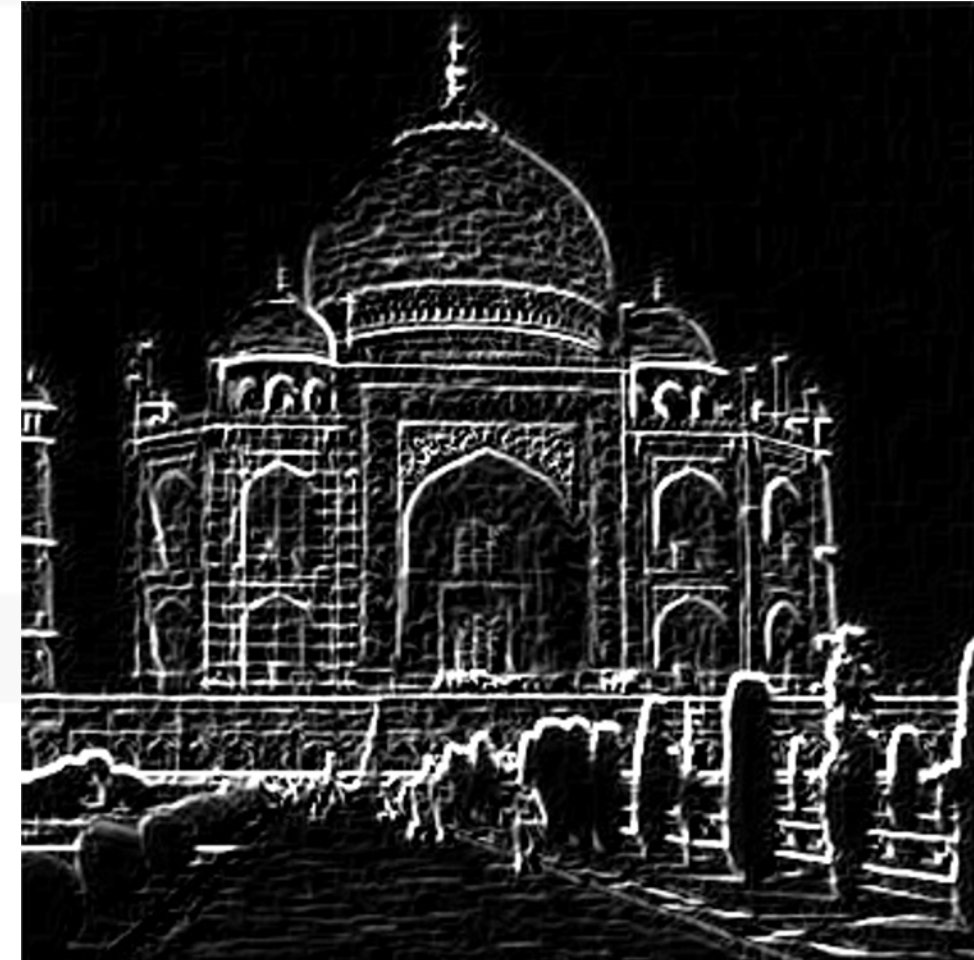
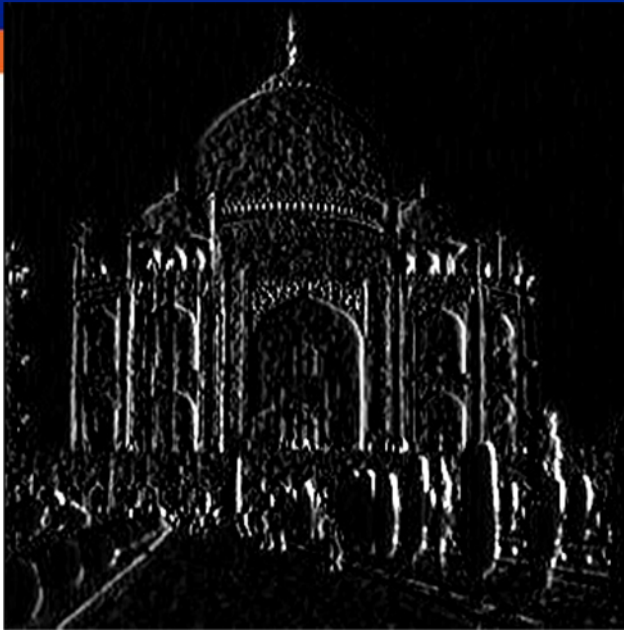
Vertical "Derivative"

Kernel

1	2	1
0	0	0
-1	-2	-1

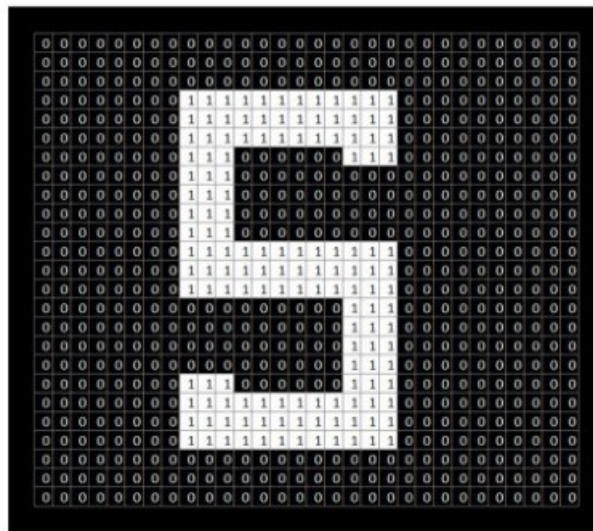


Overlay them to get **Sobel Operator**

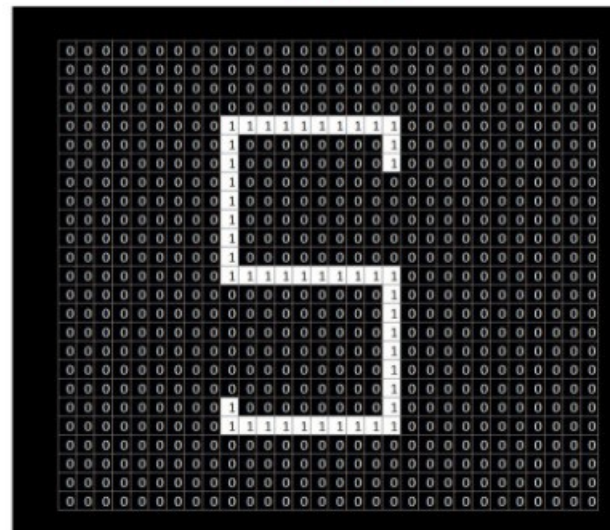


- **Dilation** – Adds pixels to the boundaries of objects in an image
- **Erosion** – Removes pixels at the boundaries of objects in an image
- **Opening** – Erosion followed by dilation
- **Closing** – Dilation followed by erosion

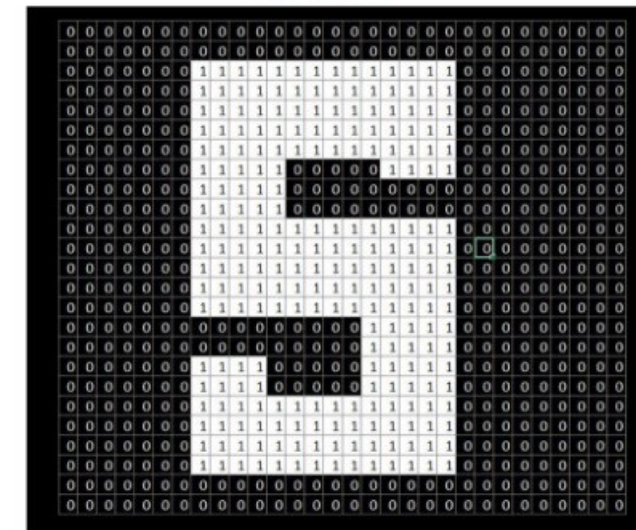
Original



Erosion



Dilation



Introduction to Convolutional Neural Networks (CNNs)

Layer Types

There are many types of layers used to build Convolutional Neural Networks, but the ones you are most likely to encounter include:

- Convolutional (CONV)
- Activation (ACT or RELU, where we use the same or the actual activation function)
- Pooling (POOL)
- Fully connected (FC)
- Dropout (DO)

Common Activation Functions

There are different activation functions

- sigmoid()
- ReLU()
- tanh()
- atan()
- softplus()
- identity()
- step-function()
- ...

ReLU is often used

Illustration

Name	Plot	Equation	Derivative
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parameteric Rectified Linear Unit (PReLU) [2]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU) [3]		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$

[Courtesy of S. Sharma]

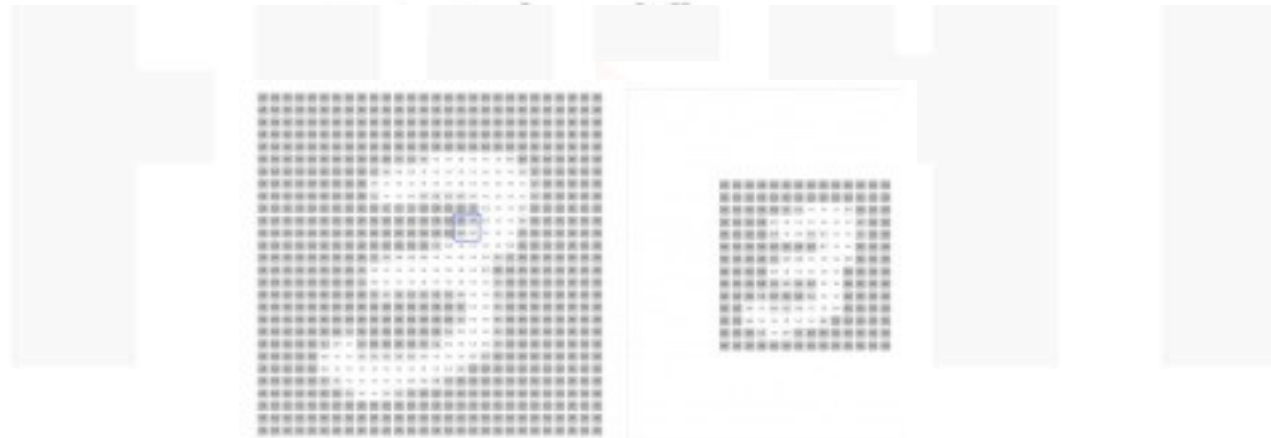
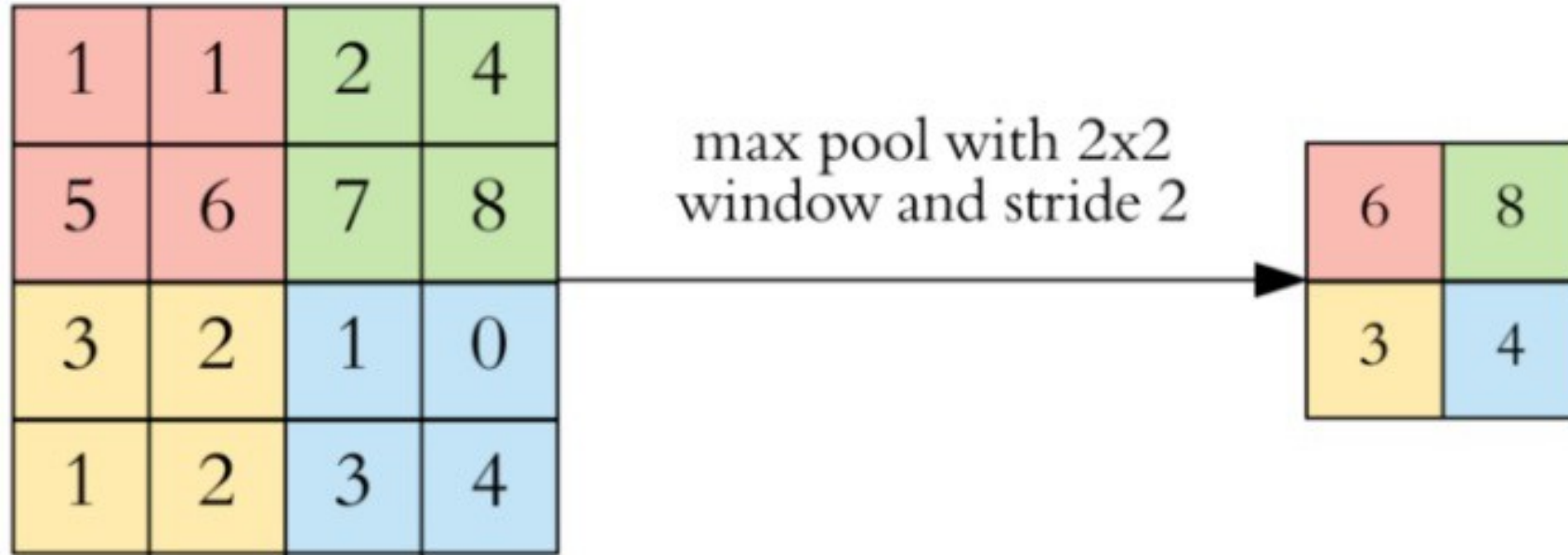
Introduction to Convolutional Neural Networks (CNNs)

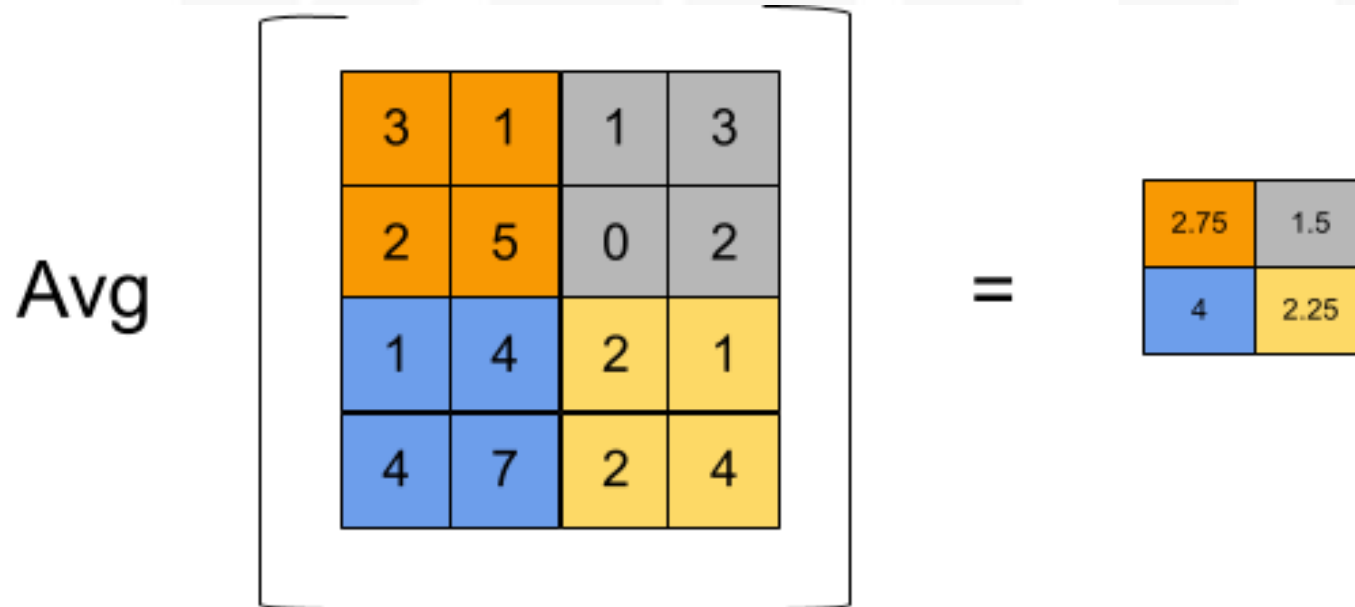
Layer Types

There are many types of layers used to build Convolutional Neural Networks, but the ones you are most likely to encounter include:

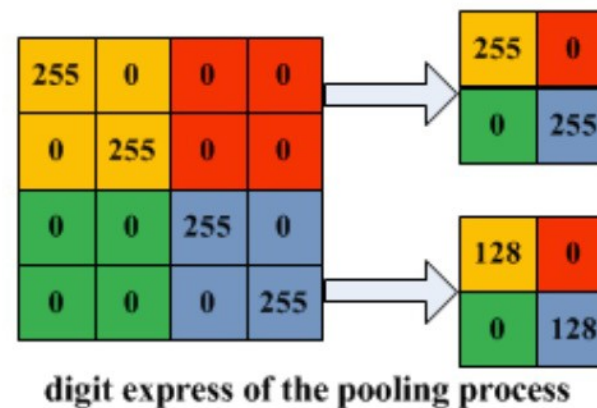
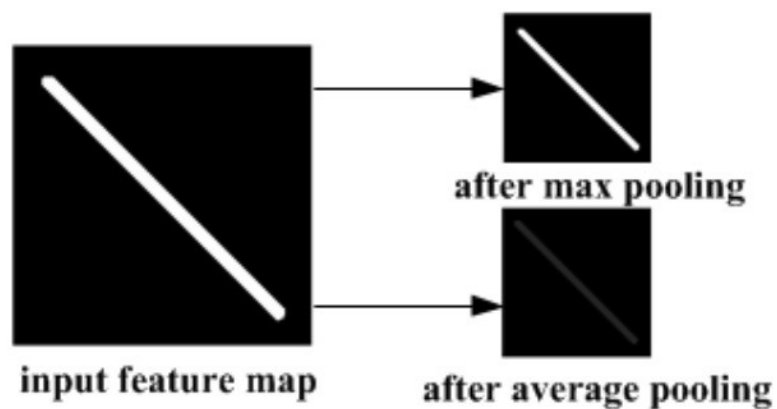
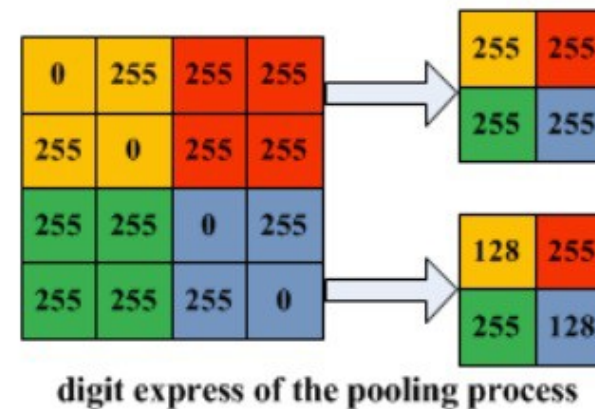
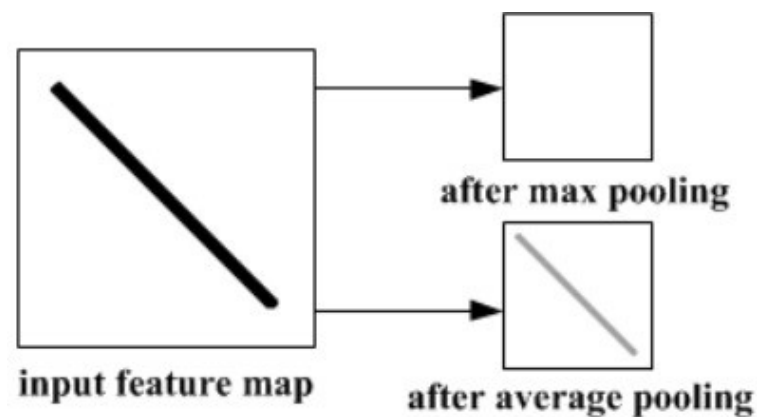
- Convolutional (CONV)
- Activation (ACT or RELU, where we use the same or the actual activation function)
- Pooling (POOL)
- Fully connected (FC)
- Dropout (DO)

Max Pooling





Kernel =2 strides =2



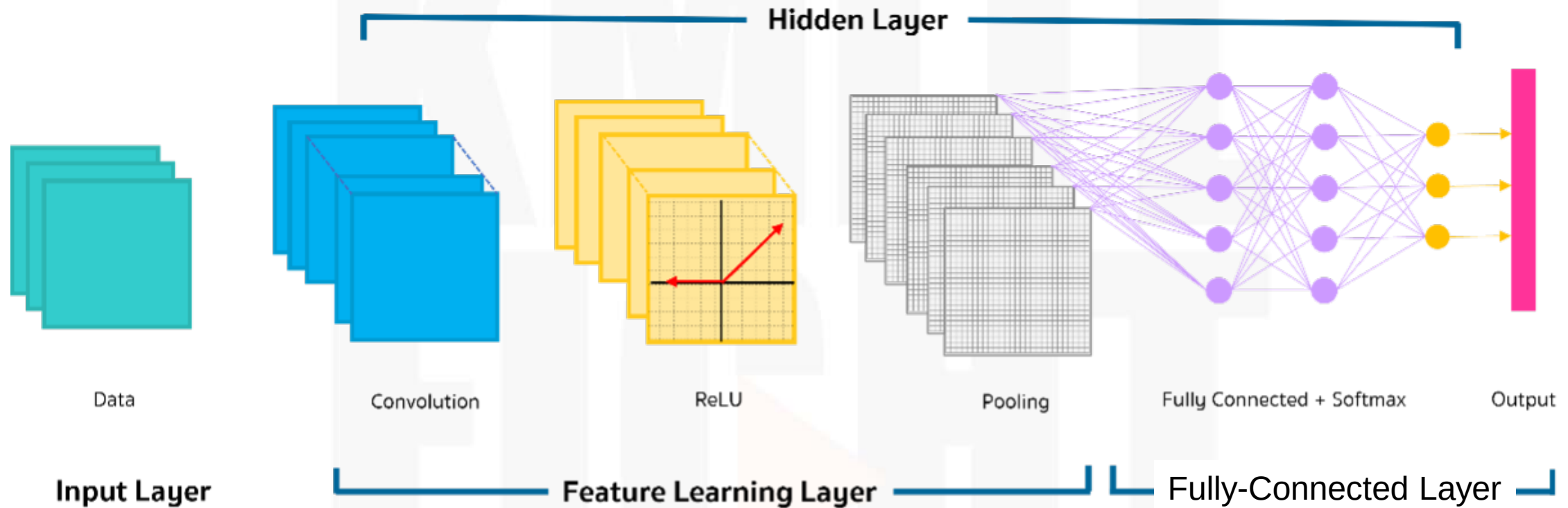
Introduction to Convolutional Neural Networks (CNNs)

Layer Types

There are many types of layers used to build Convolutional Neural Networks, but the ones you are most likely to encounter include:

- Convolutional (CONV)
- Activation (ACT or RELU, where we use the same or the actual activation function)
- Pooling (POOL)
- Fully connected (FC)
- Dropout (DO)

Introduction to Convolutional Neural Networks (CNNs)



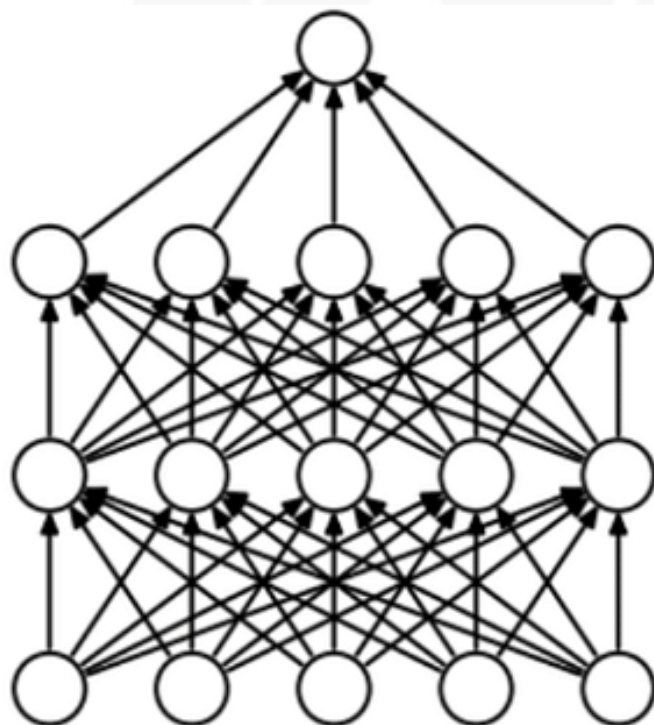
Introduction to Convolutional Neural Networks (CNNs)

Layer Types

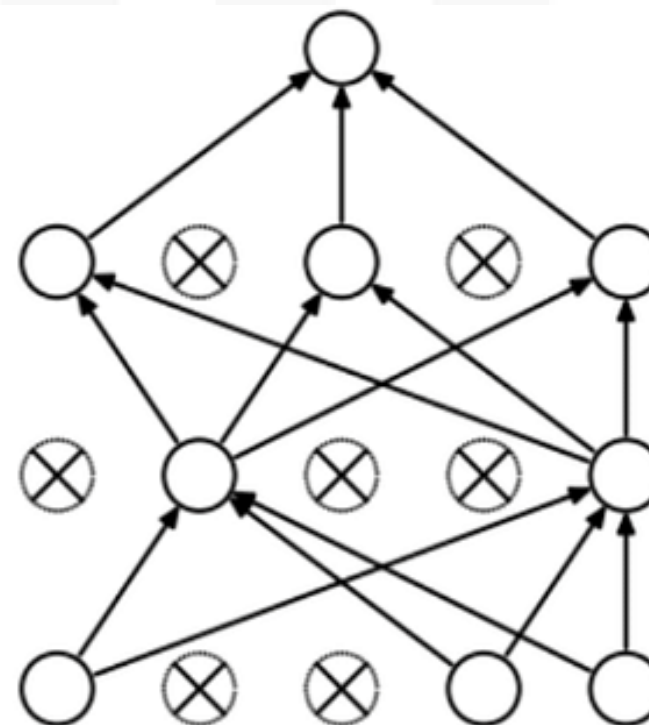
There are many types of layers used to build Convolutional Neural Networks, but the ones you are most likely to encounter include:

- Convolutional (CONV)
- Activation (ACT or RELU, where we use the same or the actual activation function)
- Pooling (POOL)
- Fully connected (FC)
- Dropout (DO)

Introduction to Convolutional Neural Networks (CNNs)



(a) Standard Neural Net



(b) After applying dropout.

source : <https://markojerkic.com/what-is-dropout-deep-learning/>

Introduction to Convolutional Neural Networks (CNNs)

Epoch

One Epoch is when an Entire dataset is passed forward and backward through the neural network only once

Introduction to Convolutional Neural Networks (CNNs)

Batch size



Introduction to Convolutional Neural Networks (CNNs)

Batch size



Introduction to Convolutional Neural Networks (CNNs)

Iterations

Iterations is the number of batches needed to complete one epoch

ตัวอย่าง MNIST dataset

จำนวน training dataset = 55,000 รูป

✓ จำนวน Dataset ที่ใช้

ถ้าเราแบ่ง batch size = 200 รูป

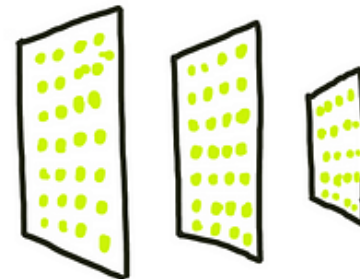
เราเป็นคนกำหนด ค่า batch size

แสดงว่าเราจะต้องใช้ $\frac{55,000}{200} = 275$ iterations ถ้าจะทำให้ train ครบ 1 epoch

มาจากการคำนวณ

55,000 รูป

0	0	0	0	0	0
1	1	1	1	1	1
2	2	2	2	2	2
3	3	3	3	3	3
4	4	4	4	4	4
5	5	5	5	5	5
6	6	6	6	6	6
7	7	7	7	7	7
8	8	8	8	8	8
9	9	9	9	9	9



→ output

@model.predict

Introduction to Convolutional Neural Networks (CNNs)

Classification v.s. Regression

Classification

- Predict which category an item belongs to based on labeled examples of known items
- Use in classify a new data

Regression

- Predict a continuous value target.
- Linear regression predicts a numeric value
- Logistic regression predicts a probability

Introduction to Convolutional Neural Networks (CNNs)

Examples of classification

- Credit card fraud detection (fraud, not fraud)
- Credit card application (good credit, bad credit)
- Email spam detection (spam, not spam)
- Text sentiment analysis (happy, not happy)
- Predicting patient risk (high risk patient, low risk patient)
- Classifying a tumor as malignant or not

Examples of regression

- Predicting the price of the house.
- Predicting age of a person
- Predicting the stock price for tomorrow.
- Predicting next year GDP.

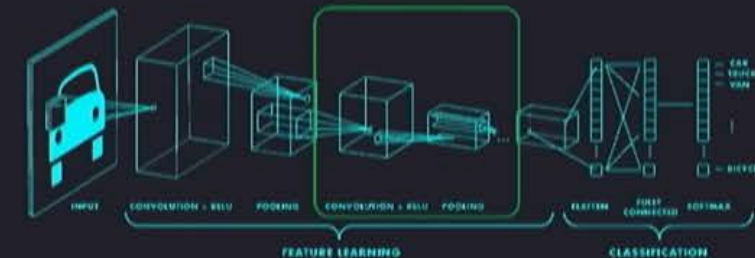
Putting it all together

```
import tensorflow as tf

def generate_model():
    model = tf.keras.Sequential([
        # first convolutional layer
        tf.keras.layers.Conv2D(32, filter_size=3, activation='relu'),
        tf.keras.layers.MaxPool2D(pool_size=2, strides=2),

        # second convolutional layer
        tf.keras.layers.Conv2D(64, filter_size=3, activation='relu'),
        tf.keras.layers.MaxPool2D(pool_size=2, strides=2),

        # fully connected classifier
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(1024, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax') # 10 outputs
    ])
    return model
```



Ref

[Convolutional Neural Network. Learn Convolutional Neural Network from... | by dshahid380 | Towards Data Science](#)

[Convolutional Neural Networks \(CNNs\) and Layer Types – PyImageSearch](#)

[Convolutional Neural Network: A Complete Guide \(learnopencv.com\)](#)

[\[Student\] Shopee Code League - Product Detection | Kaggle](#)

Lab 3

Lab OCR